

CloudExify Cybersecurity Internship

Project 03

Web Application & Penetration Testing

❖ Prepared By

Javeria Khan

❖ Reg. No

CX-INT-2026-CYB-0525

❖ Program

BS Computer Science

❖ University

University of Technology
Nowshera

❖ Email

jiya04229@gmail.com

❖ Internship Organization

CloudExify

➤ TABLE OF CONTENTS

<u>Sr. No.</u>	<u>Contents</u>	<u>Page No.</u>
01	Cover Page	01
02	Table of Contents	02
03	Introduction	08
04	Objectives	08
05	Tools Used	09
06	Environment Setup	11
07	Updating Kali Linux Package Repository	12
08	Installation of Required DVWA Dependencies	13
09	Successful Completion of DVWA Environment Dependencies Installation	14

10	Downloading DVWA into the Apache Web Server Directory	14
11	Downloading DVWA Application	15
12	Extracting DVWA Application Files	16
13	Configuring DVWA File Ownership and Permissions	18
14	Apache Web Server Status	18
15	Apache HTTP Server Running Successfully	19
16	MariaDB Database Server Status	20
17	DVWA Configuration File Error	21
18	DVWA Configuration File Created Successfully	22
19	DVWA Database Configuration File	23
20	MariaDB Database Server Started Successfully	24

21	MariaDB Database Console Accessed Successfully	24
22	DVWA Database Created Successfully	25
23	DVWA Database User Created Successfully	26
24	DVWA Database User Privileges Granted Successfully	27
25	DVWA Database User Privileges Granted Successfully	27
26	DVWA Login Page Successfully Accessed	28
27	DVWA Database Setup and Configuration Check	29
28	DVWA Login Page After Database Setup	30
29	Successful Login and DVWA Home Page	31
30	Burp Suite Community Edition Main Interface	32
31	Firefox Manual Proxy Configuration for Burp Suite	33
32	Burp Suite HTTP Request Interception Using the Forward Option	34

33	DVWA SQL Injection Testing Interface	35
34	SQL Injection Baseline Test Result	36
35	DVWA Security Level Configuration	37
36	Successful SQL Injection Test Result	38
	Successful XSS Alert Popup	39
37	Reflected Output After XSS Test	39
38	XSS Test Result at Medium Security Level	40
39	XSS Test Using Image Event Handler	41
40	Successful XSS Execution Using SVG Event Handler	42
41	XSS Test Page Output After Alert	43
42	CSRF Vulnerability Testing Interface	44
43	Successful CSRF Password Change	45
44	CSRF Request Captured in Burp Suite HTTP History	46

45	CSRF Password Change Request Captured in Burp Suite	47
46	DVWA Password Change Page Displayed in Burp Suite Response	48
47	DVWA Authentication / Brute Force Testing Interface	49
48	Successful Brute Force Login Result	50
49	CSRF Page with Pre-filled Current Password	51
50	Reflected Cross-Site Scripting (XSS) Testing	51
51	File Upload Vulnerability – Successful File Upload	52
52	Security Findings and Recommendations	53
53	Finding: SQL Injection	54
54	Finding: Cross-Site Scripting (XSS)	54
55	Finding: Cross-Site Request Forgery (CSRF)	55
56	Finding: Session and Authentication Security	55

57	Overall Recommendation	56
58	Final Security Summary and Conclusion	56
59	Conclusion	56

Web Application and Penetration Testing

➤ **Introduction**

Web applications are an important part of modern digital services, but vulnerabilities in web applications can expose sensitive information and allow unauthorized users to perform unintended actions. Therefore, security testing and penetration testing are important practices for identifying weaknesses before they can be exploited in real-world environments.

This project focuses on **Web Application and Penetration Testing** using a controlled and authorized laboratory environment. The main target of the practical testing was **Damn Vulnerable Web Application (DVWA)**, a deliberately vulnerable web application designed for cybersecurity education and security testing practice.

The project was performed in a local **Kali Linux** environment. Different security testing techniques were used to understand common web application vulnerabilities and their possible security impact. The testing included vulnerability identification, controlled exploitation, evidence collection, observation of application behavior, and analysis of possible risks.

The project also provided practical experience with commonly used penetration-testing tools such as **Burp Suite, Nmap, Nessus, and browser-based testing tools**. Burp Suite was used to inspect and analyze HTTP requests and responses, while Nmap was used for basic network and service discovery. Nessus was used to perform vulnerability assessment and identify potential security weaknesses in the target environment.

The practical activities were carried out only against the intentionally vulnerable DVWA laboratory environment. This ensured that the testing remained within an authorized and controlled setup while providing hands-on experience with real-world web security testing methodologies.

Overall, the project helped develop an understanding of how web application vulnerabilities can be discovered, analyzed, documented, and assessed from a security perspective. It also demonstrated the importance of applying appropriate security controls and remediation techniques to reduce the risk associated with vulnerable web applications.

➤ **Objectives**

The main objectives of this project are:

1. **To understand web application security** and the importance of protecting web applications from common security threats.
2. **To set up a controlled penetration-testing environment** using Kali Linux and DVWA for authorized security testing.
3. **To identify common web application vulnerabilities** through practical testing and security assessment techniques.

4. **To understand the OWASP Top 10 vulnerabilities** and their potential impact on web applications.
5. **To perform vulnerability testing** against the intentionally vulnerable DVWA application in a controlled environment.
6. **To analyze HTTP requests and responses** and understand how web applications communicate with clients and servers.
7. **To use Burp Suite** for intercepting, inspecting, and analyzing web application traffic during authorized security testing.
8. **To perform SQL Injection testing** and understand how improper input validation can affect database-driven applications.
9. **To perform Cross-Site Scripting (XSS) testing** and understand the security risks associated with unsafe user input.
10. **To analyze Cross-Site Request Forgery (CSRF)** and understand how unauthorized actions can potentially be triggered through vulnerable web applications.
11. **To perform authentication and session security testing** and identify weaknesses related to login mechanisms and session management.
12. **To use Nmap for network and service discovery** and identify available services within the authorized testing environment.
13. **To use Nessus for vulnerability assessment** and understand how automated vulnerability scanners can assist in identifying security weaknesses.
14. **To collect screenshots and testing evidence** for each major practical activity performed during the project.
15. **To assess the severity and potential impact of identified vulnerabilities** and understand their security implications.
16. **To provide suitable remediation recommendations** for the identified security issues.
17. **To improve practical penetration-testing skills** by applying cybersecurity concepts in a safe and authorized laboratory environment.
18. **To prepare professional security findings** using the structure of Finding, Evidence, Observation, Risk, Impact, and Recommendation.

➤ **Tools and Technologies Used**

3.1 Kali Linux

Kali Linux was used as the primary penetration-testing operating system. It provides a large collection of cybersecurity and penetration-testing utilities that can be used for security assessment, reconnaissance, vulnerability analysis, and web application testing.

DVWA (Damn Vulnerable Web Application)

DVWA was used as the intentionally vulnerable target application. It provides different security challenges that allow students and security professionals to safely practice identifying and understanding common web application vulnerabilities.

Burp Suite

Burp Suite was used for web application traffic analysis and penetration testing. It allows HTTP requests and responses to be inspected and analyzed during authorized testing. It was particularly useful for observing how the browser communicates with the DVWA application and for understanding the behavior of different security-testing activities.

Nmap

Nmap (Network Mapper) was used for network and service discovery. It helps identify hosts, open ports, running services, and service versions within an authorized testing environment. This information can be useful during the reconnaissance stage of a penetration test.

Apache2

Apache2 was used as the web server for hosting the DVWA application locally. It processes HTTP requests and provides the web application through the local testing environment.

MariaDB

MariaDB was used as the database server supporting the DVWA application. It stores and manages the application's database information required for the web application's functionality and security-testing exercises.

PHP

PHP was required to run the DVWA web application because DVWA is a PHP-based application. It enables the server-side processing required by the application.

Firefox Browser

Mozilla Firefox was used as the client-side web browser for accessing DVWA and performing web application security testing. It was also used in combination with Burp Suite for analyzing web traffic during the practical exercises.

VirtualBox

Oracle VirtualBox was used to provide the virtualized laboratory environment. It allowed the security-testing environment to be isolated from the main operating system and provided a controlled setup for cybersecurity experimentation.

➤ **Overall Project Outcome**

The project provided practical experience in web application security assessment and penetration testing. The controlled DVWA environment allowed common vulnerabilities to be studied without targeting real-world systems. The use of tools such as Burp Suite, Nmap, and Nessus

helped demonstrate different stages of security testing, including reconnaissance, traffic analysis, vulnerability identification, and security assessment.

The project also improved understanding of how vulnerabilities can affect the confidentiality, integrity, and availability of web applications. The findings obtained during testing were documented with supporting screenshots and security observations, followed by appropriate recommendations for improving application security.

➤ **Kali Linux Lab Environment**

Screen shot



Purpose:

To establish Kali Linux as the primary environment for conducting authorized security testing of the DVWA web application.

Objective:

- To prepare the Kali Linux environment for web security testing.
- To run the required services such as Apache and MariaDB.
- To provide a controlled environment for performing DVWA vulnerability tests.

Environment Details:

Component	Details
Operating System	Kali Linux
Virtualization Platform	Oracle VirtualBox
Virtual Machine Name	Jiya
C	Virtualized Security Testing Lab
Primary Use	Web Application Security Testing

Target Application	DVWA
Testing Scope	Local/Authorized Lab Environment

Description:

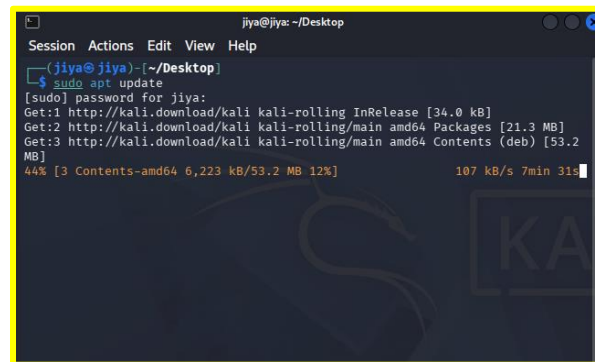
Kali Linux was used as the main testing platform for the project. The required web server, database services, DVWA application, and security testing tools were configured in the Kali Linux environment. All testing activities were performed locally in a controlled lab environment.

Result:

The Kali Linux lab environment was successfully prepared and used for the DVWA security assessment. The required services and testing tools were available for performing the planned vulnerability tests.

➤ Updating Kali Linux Package Repository

Screen shot



```

jiya@jiya: ~/Desktop
Session Actions Edit View Help
(jiya@jiya)~[~/Desktop]
$ sudo apt update
[sudo] password for jiya:
Get:1 http://kali.download/kali kali-rolling InRelease [34.0 kB]
Get:2 http://kali.download/kali kali-rolling/main amd64 Packages [21.3 MB]
Get:3 http://kali.download/kali kali-rolling/main amd64 Contents (deb) [53.2 MB]
44% [3 Contents-amd64 6,223 kB/53.2 MB 12%] 107 kB/s 7min 31s

```

Purpose:

To update the local package information so the system can access the latest available package versions required for the DVWA lab environment.

Objective:

To ensure that Kali Linux has up-to-date package repository information before installing the web server, database server, PHP components, and other DVWA dependencies.

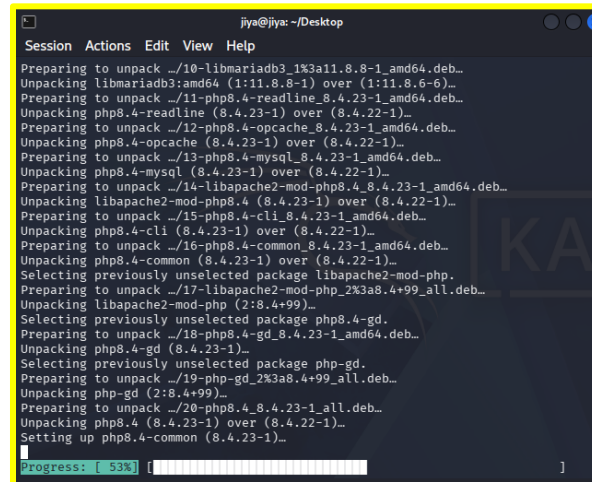
Description:

The `sudo apt update` command was executed in the Kali Linux terminal. The command refreshed the package lists from the configured repositories and prepared the system for installing the required DVWA components.

Result:

The package repository update completed successfully. Kali Linux is now ready for the next stage of the DVWA setup.

- **Installation of Required DVWA Dependencies**
- **Screen shot**



```
jiya@jiya: ~/Desktop
Session Actions Edit View Help
Preparing to unpack ./10-libmariadb3_10.3.11-1_amd64.deb...
Unpacking libmariadb3:amd64 (1:11.8.8-1) over (1:11.8.6-6)...
Preparing to unpack ./11-php8.4-readline_8.4.23-1_amd64.deb...
Unpacking php8.4-readline (8.4.23-1) over (8.4.22-1)...
Preparing to unpack ./12-php8.4-opcache_8.4.23-1_amd64.deb...
Unpacking php8.4-opcache (8.4.23-1) over (8.4.22-1)...
Preparing to unpack ./13-php8.4-mysql_8.4.23-1_amd64.deb...
Unpacking php8.4-mysql (8.4.23-1) over (8.4.22-1)...
Preparing to unpack ./14-libapache2-mod-php8.4_8.4.23-1_amd64.deb...
Unpacking libapache2-mod-php8.4 (8.4.23-1) over (8.4.22-1)...
Preparing to unpack ./15-php8.4-cli_8.4.23-1_amd64.deb...
Unpacking php8.4-cli (8.4.23-1) over (8.4.22-1)...
Preparing to unpack ./16-php8.4-common_8.4.23-1_amd64.deb...
Unpacking php8.4-common (8.4.23-1) over (8.4.22-1)...
Selecting previously unselected package libapache2-mod-php.
Preparing to unpack ./17-libapache2-mod-php_2%3a8.4+99_all.deb...
Unpacking libapache2-mod-php (2:8.4+99)...
Selecting previously unselected package php8.4-gd.
Preparing to unpack ./18-php8.4-gd_8.4.23-1_amd64.deb...
Unpacking php8.4-gd (8.4.23-1)...
Selecting previously unselected package php-gd.
Preparing to unpack ./19-php-gd_2%3a8.4+99_all.deb...
Unpacking php-gd (2:8.4+99)...
Preparing to unpack ./20-php8.4_8.4.23-1_all.deb...
Unpacking php8.4 (8.4.23-1) over (8.4.22-1)...
Setting up php8.4-common (8.4.23-1)...
Progress: [ 53% ]
```

Purpose:

To install the software components required to create the local DVWA testing environment.

Objective:

- To install the Apache web server for hosting DVWA.
- To install MariaDB for database management.
- To install PHP and required PHP extensions for DVWA functionality.
- To ensure all necessary dependencies are available for the DVWA setup.

Description:

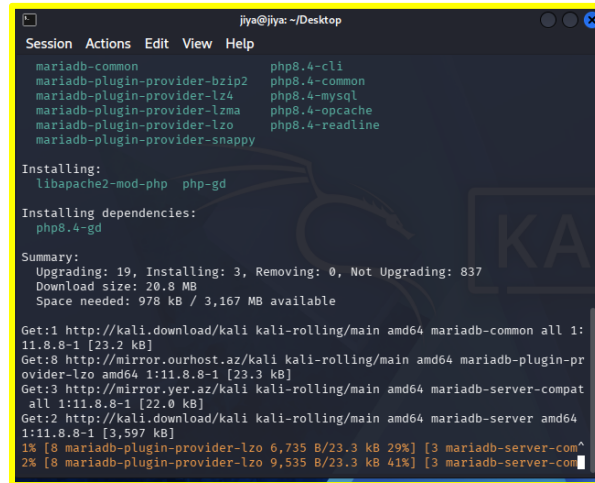
The required DVWA dependencies were installed in the Kali Linux environment. These included the Apache web server, MariaDB database server, PHP, PHP-MySQL support, PHP-GD, and the Apache PHP module. These components are necessary for running DVWA locally.

Result:

The required web server, database, PHP, and supporting components were installed successfully.

➤ Successful Completion of DVWA Environment Dependencies Installation

Screen shot



```
jya@jya: ~/Desktop
Session Actions Edit View Help

mariadb-common php8.4-cli
mariadb-plugin-provider-bzip2 php8.4-common
mariadb-plugin-provider-lz4 php8.4-mysql
mariadb-plugin-provider-lzma php8.4-opcache
mariadb-plugin-provider-lzo php8.4-readline
mariadb-plugin-provider-snappy

Installing:
libapache2-mod-php php-gd

Installing dependencies:
php8.4-gd

Summary:
Upgrading: 19, Installing: 3, Removing: 0, Not Upgrading: 837
Download size: 20.8 MB
Space needed: 978 kB / 3,167 MB available

Get:1 http://kali.download/kali kali-rolling/main amd64 mariadb-common all 1:11.8.8-1 [23.2 kB]
Get:8 http://mirror.ourhost.az/kali kali-rolling/main amd64 mariadb-plugin-provider-lzo amd64 1:11.8.8-1 [23.3 kB]
Get:3 http://mirror.yer.az/kali kali-rolling/main amd64 mariadb-server-compat all 1:11.8.8-1 [22.0 kB]
Get:2 http://kali.download/kali kali-rolling/main amd64 mariadb-server amd64 1:11.8.8-1 [3,597 kB]
1% [8 mariadb-plugin-provider-lzo 6,735 B/23.3 kB 29%] [3 mariadb-server-com^
2% [8 mariadb-plugin-provider-lzo 9,535 B/23.3 kB 41%] [3 mariadb-server-com
```

Purpose:

To verify that the required packages were successfully configured after installation.

Description:

The terminal shows the configuration and processing of Apache PHP modules, MariaDB components, and other required packages. The terminal prompt returned after processing completed, indicating that the installation process finished successfully.

Result:

The required DVWA dependencies were successfully installed and configured, and the Kali Linux environment is ready for the DVWA application setup.

➤ Downloading DVWA into the Apache Web Server Directory

Screen shot

```
Installing new version of config file /etc/mysql/mariadb.conf.d/50-server.cnf...
...
mariadb.service is a disabled or a static unit not running, not starting it.
Setting up libapache2-mod-php8.4 (8.4.23-1)...
libapache2-mod-php8.4: not switching MPM - already enabled
Setting up libapache2-mod-php (2:8.4+99)...
Setting up php8.4 (8.4.23-1)...
Setting up mariadb-server-compat (1:11.8.8-1)...
Processing triggers for man-db (2.13.1-1)...
Processing triggers for kali-menu (2026.2.6)...
Processing triggers for libc-bin (2.42-16)...
Processing triggers for php8.4-cli (8.4.23-1)...
Processing triggers for libapache2-mod-php8.4 (8.4.23-1)...

(jiya@jiya)~[~/Desktop]
$ cd /var/www/html

(jiya@jiya)~[~/var/www/html]
$ sudo git clone https://github.com/digininja/DVWA.get dvwa
[sudo] password for jiya:
Cloning into 'dvwa' ...
Username for 'https://github.com': █
```

Purpose:

To obtain the authorized DVWA training application and place it in the local Apache web-server directory.

Objective:

- To download the DVWA application into the Kali Linux environment.
- To place DVWA in the Apache web-server directory.
- To prepare the application for local security testing.

Description:

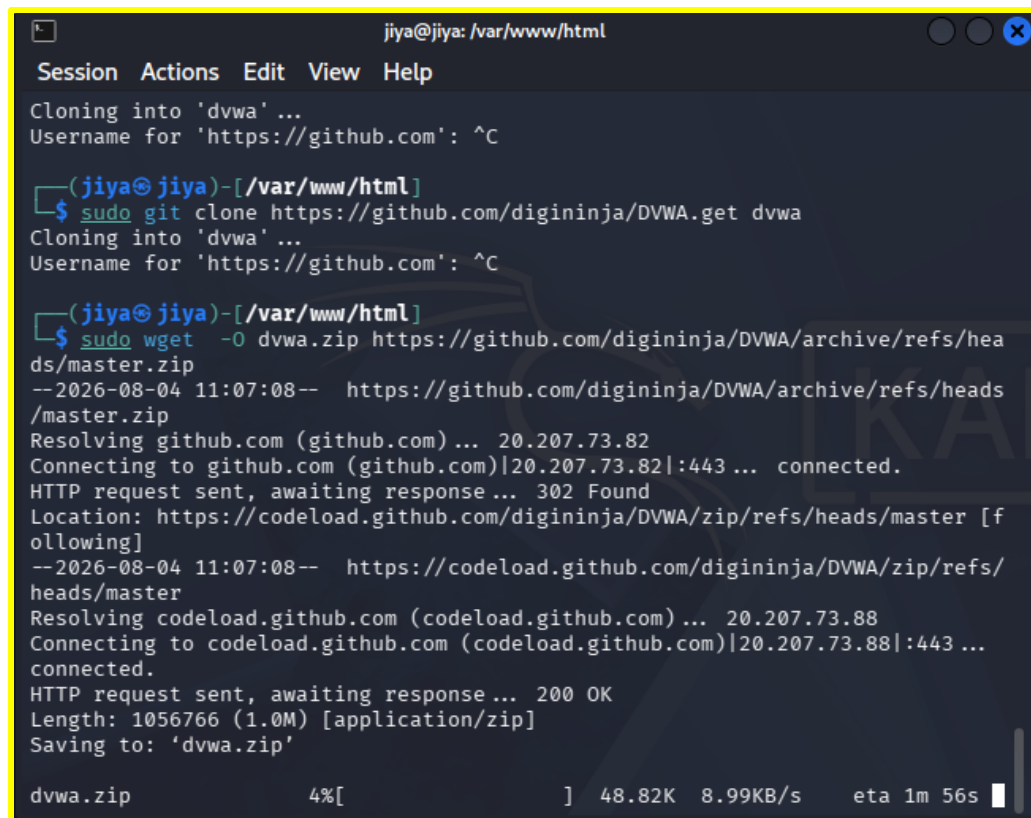
The DVWA application was downloaded and placed in the Apache web-server directory (/var/www/html/). This allowed the application to be hosted locally through the Apache web server and prepared it for subsequent configuration and vulnerability testing.

Result:

DVWA was successfully obtained and prepared for local web application security testing.

➤ **Downloading DVWA Application**

Screen shot



```
jiya@jiya: /var/www/html
Session Actions Edit View Help
Cloning into 'dvwa' ...
Username for 'https://github.com': ^C

(jiya@jiya)-[/var/www/html]
$ sudo git clone https://github.com/digininja/DVWA.get dvwa
Cloning into 'dvwa' ...
Username for 'https://github.com': ^C

(jiya@jiya)-[/var/www/html]
$ sudo wget -O dvwa.zip https://github.com/digininja/DVWA/archive/refs/heads/master.zip
--2026-08-04 11:07:08-- https://github.com/digininja/DVWA/archive/refs/heads/master.zip
Resolving github.com (github.com)... 20.207.73.82
Connecting to github.com (github.com)|20.207.73.82|:443 ... connected.
HTTP request sent, awaiting response... 302 Found
Location: https://codeload.github.com/digininja/DVWA/zip/refs/heads/master [following]
--2026-08-04 11:07:08-- https://codeload.github.com/digininja/DVWA/zip/refs/heads/master
Resolving codeload.github.com (codeload.github.com)... 20.207.73.88
Connecting to codeload.github.com (codeload.github.com)|20.207.73.88|:443 ... connected.
HTTP request sent, awaiting response... 200 OK
Length: 1056766 (1.0M) [application/zip]
Saving to: 'dvwa.zip'

dvwa.zip          4%[          ] 48.82K  8.99KB/s  eta 1m 56s
```

Purpose:

To obtain the DVWA application files for the controlled local web-security testing environment.

Objective:

To successfully download the DVWA package into the Apache web-server directory.

Description:

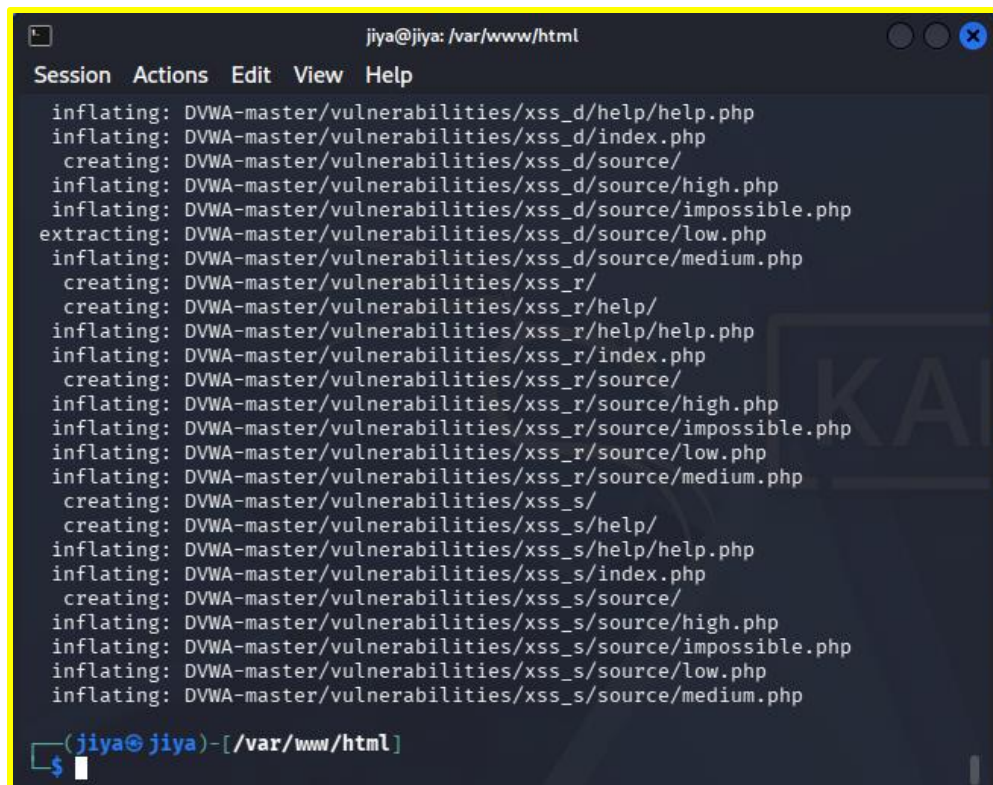
The DVWA ZIP archive was downloaded successfully using `wget`. The terminal shows an HTTP 200 OK response and a completed download of the `dvwa.zip` file.

Result:

The DVWA package was successfully downloaded and is ready for extraction and configuration.

➤ **Extracting DVWA Application Files**

Screen shot

A terminal window titled 'jiya@jiya: /var/www/html' with a menu bar (Session, Actions, Edit, View, Help). It displays the output of a command, showing a list of files being inflated and created within the DVWA-master directory structure. The files are organized into subdirectories like xss_d, xss_r, and xss_s, each containing help, index, and source files. The terminal ends with a prompt '(jiya@jiya)-[/var/www/html]' and a cursor.

```
jiya@jiya: /var/www/html
Session Actions Edit View Help
inflating: DVWA-master/vulnerabilities/xss_d/help/help.php
inflating: DVWA-master/vulnerabilities/xss_d/index.php
creating: DVWA-master/vulnerabilities/xss_d/source/
inflating: DVWA-master/vulnerabilities/xss_d/source/high.php
inflating: DVWA-master/vulnerabilities/xss_d/source/impossible.php
extracting: DVWA-master/vulnerabilities/xss_d/source/low.php
inflating: DVWA-master/vulnerabilities/xss_d/source/medium.php
creating: DVWA-master/vulnerabilities/xss_r/
creating: DVWA-master/vulnerabilities/xss_r/help/
inflating: DVWA-master/vulnerabilities/xss_r/help/help.php
inflating: DVWA-master/vulnerabilities/xss_r/index.php
creating: DVWA-master/vulnerabilities/xss_r/source/
inflating: DVWA-master/vulnerabilities/xss_r/source/high.php
inflating: DVWA-master/vulnerabilities/xss_r/source/impossible.php
inflating: DVWA-master/vulnerabilities/xss_r/source/low.php
inflating: DVWA-master/vulnerabilities/xss_r/source/medium.php
creating: DVWA-master/vulnerabilities/xss_s/
creating: DVWA-master/vulnerabilities/xss_s/help/
inflating: DVWA-master/vulnerabilities/xss_s/help/help.php
inflating: DVWA-master/vulnerabilities/xss_s/index.php
creating: DVWA-master/vulnerabilities/xss_s/source/
inflating: DVWA-master/vulnerabilities/xss_s/source/high.php
inflating: DVWA-master/vulnerabilities/xss_s/source/impossible.php
inflating: DVWA-master/vulnerabilities/xss_s/source/low.php
inflating: DVWA-master/vulnerabilities/xss_s/source/medium.php
(jiya@jiya)-[/var/www/html]
```

Purpose:

To extract the downloaded DVWA archive and prepare its application files for local web-server configuration.

Objective:

To unpack the `dvwa.zip` archive inside the Apache web directory so that the DVWA application can be accessed locally.

Description:

The `sudo unzip dvwa.zip` command was executed from the `/var/www/html` directory. The archive was extracted successfully, making the DVWA application files available for the next configuration steps.

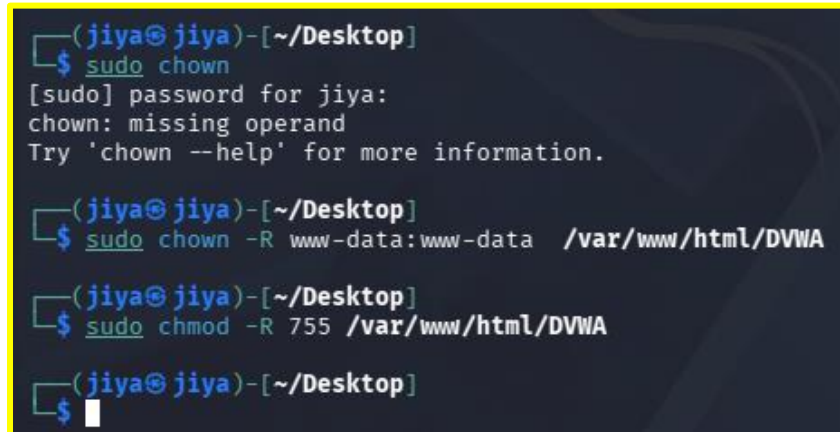
Result:

The DVWA archive was successfully extracted and the application files are now available in the local Apache web-server directory.

//////////

➤ Configuring DVWA File Ownership and Permissions

Screen shot



```
(jiya@jiya)-[~/Desktop]
$ sudo chown
[sudo] password for jiya:
chown: missing operand
Try 'chown --help' for more information.

(jiya@jiya)-[~/Desktop]
$ sudo chown -R www-data:www-data /var/www/html/DVWA

(jiya@jiya)-[~/Desktop]
$ sudo chmod -R 755 /var/www/html/DVWA

(jiya@jiya)-[~/Desktop]
$
```

Purpose:

To configure the ownership and permissions of the DVWA files for the Apache web server.

Objective:

To ensure that the web server can properly access and serve the DVWA application files.

Description:

The `chown` command assigned the DVWA directory to the `www-data` user and group used by Apache. The `chmod` command set the required permissions on the DVWA files and directories.

Commands Used:

```
sudo chown -R www-data:www-data /var/www/html/DVWA
sudo chmod -R 755 /var/www/html/DVWA
```

Result:

DVWA file ownership and permissions were successfully configured, preparing the application for Apache-based execution.

➤ Apache Web Server Status

Screen shot

```
jiya@jiya: ~$ sudo systemctl status apache2
[sudo] password for jiya:
o apache2.service - The Apache HTTP Server
   Loaded: loaded (/usr/lib/systemd/system/apache2.service; disabled; preset: disabled)
   Active: inactive (dead)
   Docs: https://httpd.apache.org/docs/2.4/

(jiya@jiya)~$ sudo systemctl start apache2

(jiya@jiya)~$ sudo systemctl status apache2
● apache2.service - The Apache HTTP Server
   Loaded: loaded (/usr/lib/systemd/system/apache2.service; disabled; preset: disabled)
   Active: active (running) since Thu 2026-08-13 10:24:48 PKT; 40s ago
   Invocation: d332a4aab88c4bc9977685d2faf82ebd
   Docs: https://httpd.apache.org/docs/2.4/
   Main PID: 7972 (apache2)
   Status: "Total requests: 0; Idle/Busy workers 100/0; Requests/sec: 0; Bytes served/sec: 0 B/Sec"
   Tasks: 6 (limit: 2280)
   Memory: 59.2M (peak: 60.1M)
   CPU: 215ms
   CGroup: /system.slice/apache2.service
           └─7972 /usr/sbin/apache2 -k start -DFOREGROUND
             └─7996 /usr/sbin/apache2 -k start -DFOREGROUND
               └─7998 /usr/sbin/apache2 -k start -DFOREGROUND
                 └─7999 /usr/sbin/apache2 -k start -DFOREGROUND
                   └─8000 /usr/sbin/apache2 -k start -DFOREGROUND
                     └─8001 /usr/sbin/apache2 -k start -DFOREGROUND

Aug 13 10:24:47 jiya systemd[1]: Starting apache2.service - The Apache HTTP Server...
Aug 13 10:24:48 jiya apachectl[7972]: AH00558: apache2: Could not reliably determine the server's fully qualified domain name, using 127.0.1.1.
Aug 13 10:24:48 jiya systemd[1]: Started apache2.service - The Apache HTTP Server.
lines 1-21/21 (END)
```

➤ Apache HTTP Server Running Successfully

```
jiya@jiya: ~/Desktop
Session Actions Edit View Help

(jiya@jiya)~$ sudo systemctl start mariadb

(jiya@jiya)~$ sudo systemctl status apache2
● apache2.service - The Apache HTTP Server
   Loaded: loaded (/usr/lib/systemd/system/apache2.service; disabled; preset: disabled)
   Active: active (running) since Thu 2026-08-06 11:42:53 PKT; 1min 49s ago
   Invocation: 2c8cbc5288834228972237afb704c21e
   Docs: https://httpd.apache.org/docs/2.4/
   Main PID: 51063 (apache2)
   Status: "Total requests: 0; Idle/Busy workers 100/0; Requests/sec: 0; Bytes served/sec: 0 B/Sec"
   Tasks: 6 (limit: 2280)
   Memory: 23.1M (peak: 23.1M)
   CPU: 166ms
   CGroup: /system.slice/apache2.service
           └─51063 /usr/sbin/apache2 -k start -DFOREGROUND
             └─51079 /usr/sbin/apache2 -k start -DFOREGROUND
               └─51080 /usr/sbin/apache2 -k start -DFOREGROUND
                 └─51082 /usr/sbin/apache2 -k start -DFOREGROUND
                   └─51083 /usr/sbin/apache2 -k start -DFOREGROUND
                     └─51084 /usr/sbin/apache2 -k start -DFOREGROUND

Aug 06 11:42:53 jiya systemd[1]: Starting apache2.service - The Apache HTTP Server...
Aug 06 11:42:53 jiya apachectl[51063]: AH00558: apache2: Could not reliably determine the server's fully qualified domain name, using 127.0.1.1.
Aug 06 11:42:53 jiya systemd[1]: Started apache2.service - The Apache HTTP Server.
lines 1-21/21 (END)
```

Purpose:

To start and verify the Apache web server required to host the DVWA application.

Objective:

To confirm that Apache is running correctly and is ready to serve the DVWA web application.

Description:

The Apache HTTP Server was started using the system service manager. The service status was checked, and the terminal displayed “**Active: active (running)**”, confirming that Apache is operating successfully.

Command Used:

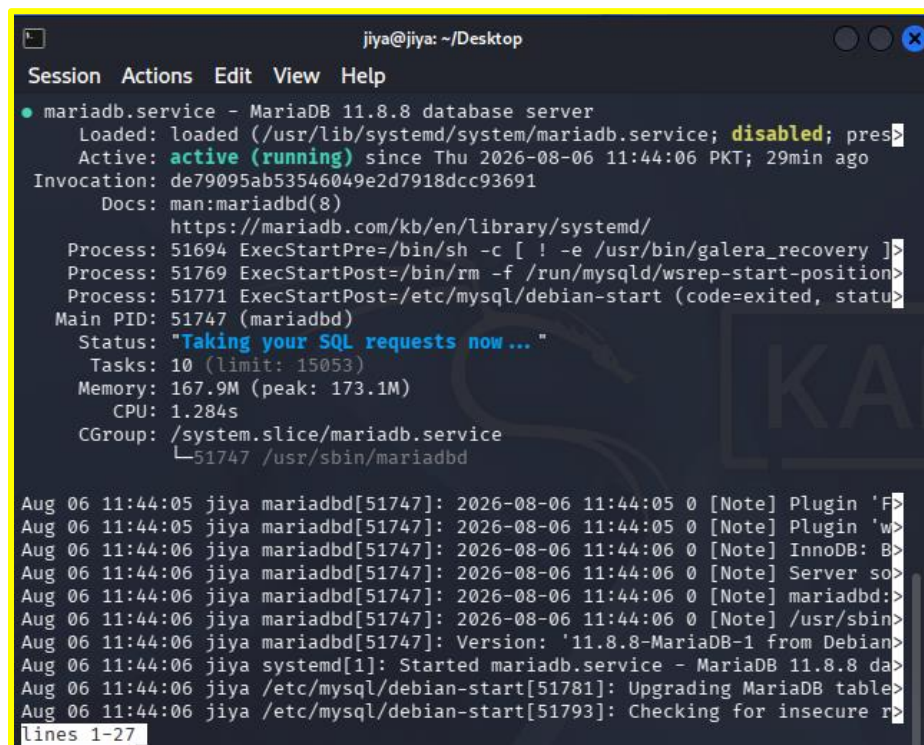
```
sudo systemctl start apache2
sudo systemctl status apache2
```

Result:

Apache HTTP Server started successfully and was confirmed to be in an active running state, providing the web-server environment required for DVWA.

➤ MariaDB Database Server Status

Screen shot



```
jiya@jiya: ~/Desktop
Session Actions Edit View Help
● mariadb.service - MariaDB 11.8.8 database server
   Loaded: loaded (/usr/lib/systemd/system/mariadb.service; disabled; pres>
   Active: active (running) since Thu 2026-08-06 11:44:06 PKT; 29min ago
   Invocation: de79095ab53546049e2d7918dcc93691
   Docs: man:mariadb(8)
        https://mariadb.com/kb/en/library/systemd/
   Process: 51694 ExecStartPre=/bin/sh -c [ ! -e /usr/bin/galera_recovery ]>
   Process: 51769 ExecStartPost=/bin/rm -f /run/mysqld/wsrep-start-position>
   Process: 51771 ExecStartPost=/etc/mysql/debian-start (code=exited, statu>
   Main PID: 51747 (mariabdd)
   Status: "Taking your SQL requests now ..."
   Tasks: 10 (limit: 15053)
   Memory: 167.9M (peak: 173.1M)
   CPU: 1.284s
   CGroup: /system.slice/mariadb.service
           └─51747 /usr/sbin/mariabdd

Aug 06 11:44:05 jiya mariabdd[51747]: 2026-08-06 11:44:05 0 [Note] Plugin 'F>
Aug 06 11:44:05 jiya mariabdd[51747]: 2026-08-06 11:44:05 0 [Note] Plugin 'w>
Aug 06 11:44:06 jiya mariabdd[51747]: 2026-08-06 11:44:06 0 [Note] InnoDB: B>
Aug 06 11:44:06 jiya mariabdd[51747]: 2026-08-06 11:44:06 0 [Note] Server so>
Aug 06 11:44:06 jiya mariabdd[51747]: 2026-08-06 11:44:06 0 [Note] mariabdd:>
Aug 06 11:44:06 jiya mariabdd[51747]: 2026-08-06 11:44:06 0 [Note] /usr/sbin>
Aug 06 11:44:06 jiya mariabdd[51747]: Version: '11.8.8-MariaDB-1 from Debian>
Aug 06 11:44:06 jiya systemd[1]: Started mariadb.service - MariaDB 11.8.8 da>
Aug 06 11:44:06 jiya /etc/mysql/debian-start[51781]: Upgrading MariaDB table>
Aug 06 11:44:06 jiya /etc/mysql/debian-start[51793]: Checking for insecure r>
lines 1-27
```

Purpose:

To start and verify the MariaDB database service required by DVWA.

Objective:

To confirm that the database server is running properly and ready for DVWA.

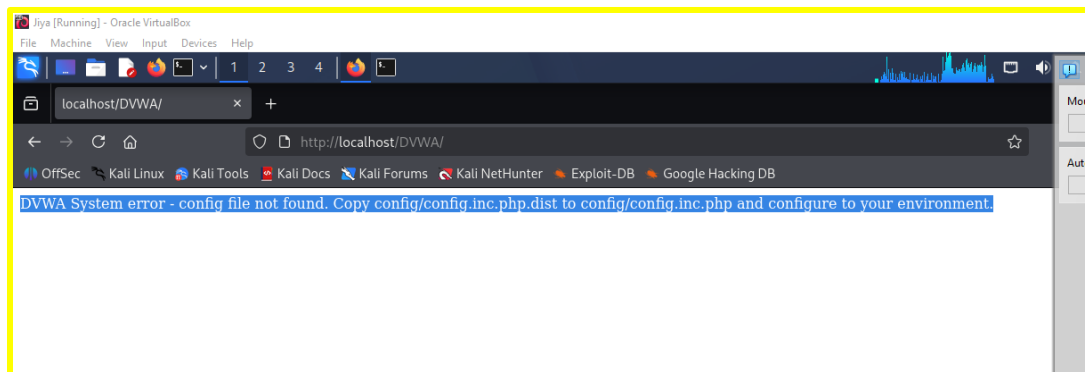
Description:

The MariaDB service status was checked through the system service manager. The “**Active: active (running)**” status confirms that MariaDB is operating successfully.

Result:

MariaDB was successfully started and verified as active, providing the database environment required by DVWA.

➤ **DVWA Configuration File Error**

Screen shot**Purpose:**

To verify that the DVWA application is accessible through Apache and identify the configuration file required for the application to run.

Objective:

To identify the missing DVWA configuration file and determine the required configuration step before continuing with the DVWA setup.

Description:

The DVWA page was successfully reached through `http://localhost/DVWA/`. However, the application displayed a “**config file not found**” message and instructed that `config/config.inc.php.dist` should be copied to `config/config.inc.php` and then configured according to the environment.

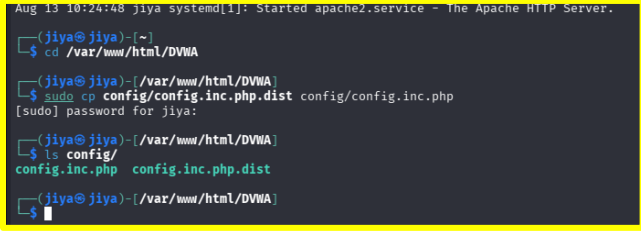
Observation:

Apache is now serving the DVWA application, but DVWA cannot proceed because its required configuration file has not yet been created.

Result:

The DVWA application was successfully accessed through the local Apache server, and the missing configuration-file issue was identified. The next step is to create and configure `config/config.inc.php`.

➤ DVWA Configuration File Created Successfully

Screen shotA terminal window screenshot with a dark background and yellow text. The terminal shows the following sequence of commands and output: 1. A system message at the top: "Aug 13 10:24:48 jiya systemd[1]: Started apache2.service - The Apache HTTP Server." 2. Prompt: "(jiya@jiya)-[~]" 3. Command: "\$ cd /var/www/html/DVWA" 4. Prompt: "(jiya@jiya)-[/var/www/html/DVWA]" 5. Command: "\$ sudo cp config/config.inc.php.dist config/config.inc.php" 6. Output: "[sudo] password for jiya:" 7. Prompt: "(jiya@jiya)-[/var/www/html/DVWA]" 8. Command: "\$ ls config/" 9. Output: "config.inc.php config.inc.php.dist" 10. Prompt: "(jiya@jiya)-[/var/www/html/DVWA]" 11. Command: "\$" followed by a cursor.**Purpose:**

To create the required DVWA configuration file so that the application can proceed with its configuration.

Objective:

To copy the default DVWA configuration template and create the required `config.inc.php` file.

Description:

The DVWA directory was accessed through the terminal, and the default configuration template was copied to create `config.inc.php`. The `ls config/` command was then used to verify the configuration directory. The screenshot shows `config.inc.php`, confirming that the required configuration file was created successfully.

Commands Used:

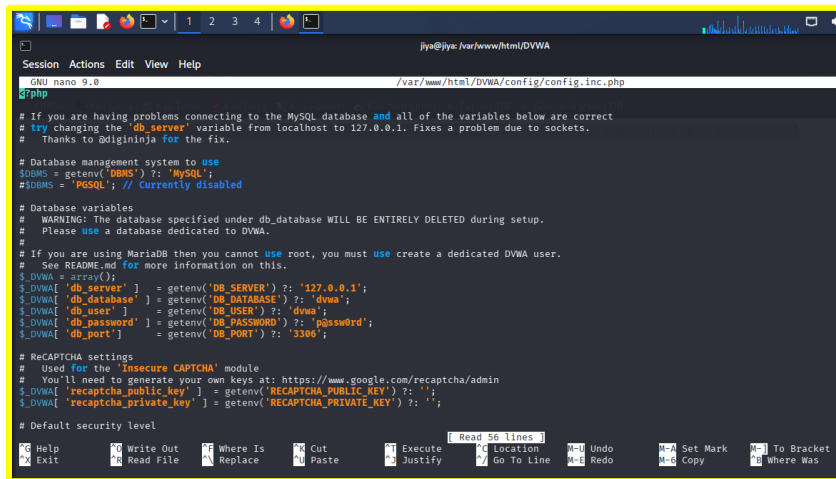
```
cd /var/www/html/DVWA
sudo cp config/config.inc.php.dist config/config.inc.php
ls config/
```

Result:

The required `config.inc.php` configuration file was successfully created and verified. DVWA is now ready for the next configuration stage.

➤ DVWA Database Configuration File

Screen shot



```
GNU nano 9.0 /var/www/html/DVWA/config/config.inc.php
$php
# If you are having problems connecting to the MySQL database and all of the variables below are correct
# try changing the 'db_server' variable from localhost to 127.0.0.1. Fixes a problem due to sockets.
# Thanks to @digininja for the fix.

# Database management system to use
$DBMS = getenv('DBMS') ? 'MySQL' :
# $DBMS = 'PGSQL'; // Currently disabled

# Database variables
# WARNING: The database specified under db_database WILL BE ENTIRELY DELETED during setup.
# Please use a database dedicated to DVWA.
#
# If you are using MariaDB then you cannot use root, you must use create a dedicated DVWA user.
# See README.md for more information on this.
$DVWA = array();
$DVWA['db_server'] = getenv('DB_SERVER') ? '127.0.0.1' :
$DVWA['db_database'] = getenv('DB_DATABASE') ? 'dvwa' :
$DVWA['db_user'] = getenv('DB_USER') ? 'dvwa' :
$DVWA['db_password'] = getenv('DB_PASSWORD') ? 'p@ssw0rd' :
$DVWA['db_port'] = getenv('DB_PORT') ? '3306' :

# ReCAPTCHA settings
# Used for the 'Insecure CAPTCHA' module
# You'll need to generate your own keys at: https://www.google.com/recaptcha/admin
$DVWA['recaptcha_public_key'] = getenv('RECAPTCHA_PUBLIC_KEY') ? '' :
$DVWA['recaptcha_private_key'] = getenv('RECAPTCHA_PRIVATE_KEY') ? '' :

# Default security level
```

Purpose:

To review the DVWA database configuration settings required for connecting the application to its database server.

Objective:

To verify the database server, database name, database user, password, and port settings defined in the DVWA configuration file.

Description:

The config.inc.php file was opened using the Nano editor. The configuration screen displays the main DVWA database variables, including DB_SERVER, DB_DATABASE, DB_USER, DB_PASSWORD, and DB_PORT. These settings are required for DVWA to communicate with the database server.

Configuration Details Visible:

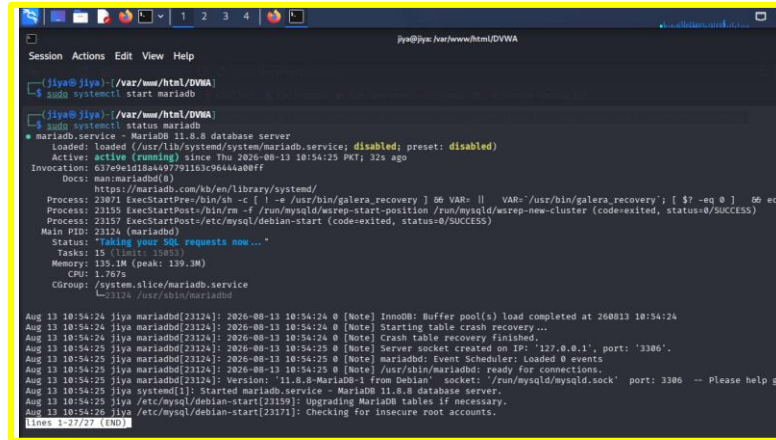
- **DB Server:** 127.0.0.1
- **Database:** dvwa
- **DB User:** dvwa
- **DB Port:** 3306

Result:

The DVWA database configuration file was successfully opened and its required database connection settings were identified. Further database configuration is required before DVWA can complete its setup.

➤ MariaDB Database Server Started Successfully

Screen shot



```
Session Actions Edit View Help
jya@jya: /var/www/html/DVWA

[jya@jya: /var/www/html/DVWA]
$ sudo systemctl start mariadb

[jya@jya: /var/www/html/DVWA]
$ sudo systemctl status mariadb
● mariadb.service - MariaDB 11.8.8 database server
   Loaded: loaded (/usr/lib/systemd/system/mariadb.service; disabled; preset: disabled)
   Active: active (running) since Thu 2026-08-13 10:54:25 PDT; 32s ago
   Invocation: 637e9e1d8a4497791163c9644a00ff
   Docs: man:mariadb(8)
        https://mariadb.com/kb/en/library/systemd/
  Process: 23871 ExecStartPre=/bin/sh -c [ ! -e /usr/bin/galera_recovery ] && VAR= || VAR="/usr/bin/galera_recovery"; [ $? -eq 0 ] && ech
  Process: 23155 ExecStartPost=/bin/rm -f /run/mysqld/wsrep-start-position /run/mysqld/wsrep-new-cluster (code=exited, status=0/SUCCESS)
  Process: 23157 ExecStartPost=/etc/mysql/debian-start (code=exited, status=0/SUCCESS)
  Main PID: 23124 (mariadbd)
  Status: "Taking your SQL requests now..."
    Tasks: 15 (limit: 1000)
  Memory: 135.1M (peak: 139.3M)
    CPU: 1.767s
  CGroup: /system.slice/mariadb.service
          └─23124 /usr/sbin/mariadbd

Aug 13 10:54:24 jya mariadbd[23124]: 2026-08-13 10:54:24 0 [Note] InnoDB: Buffer pool(s) load completed at 200813 10:54:24
Aug 13 10:54:24 jya mariadbd[23124]: 2026-08-13 10:54:24 0 [Note] Starting table crash recovery...
Aug 13 10:54:24 jya mariadbd[23124]: 2026-08-13 10:54:24 0 [Note] Crash table recovery finished.
Aug 13 10:54:25 jya mariadbd[23124]: 2026-08-13 10:54:25 0 [Note] Server socket created on IP: '127.0.0.1', port: '3306'.
Aug 13 10:54:25 jya mariadbd[23124]: 2026-08-13 10:54:25 0 [Note] mariadbd: Event Scheduler: Loaded 0 events
Aug 13 10:54:25 jya mariadbd[23124]: 2026-08-13 10:54:25 0 [Note] /usr/sbin/mariadbd: ready for connections.
Aug 13 10:54:25 jya mariadbd[23124]: Version: '11.8.8-MariaDB-1' from debian  socket: '/run/mysqld/mysqld.sock' port: 3306 -- Please help ge
Aug 13 10:54:25 jya systemd[1]: Started mariadb.service - MariaDB 11.8.8 database server.
Aug 13 10:54:25 jya /etc/mysql/debian-start[23159]: Upgrading MariaDB tables if necessary.
Aug 13 10:54:26 jya /etc/mysql/debian-start[23171]: Checking for insecure root accounts.

[lines 1-27/27 (END)]
```

Purpose:

To start and verify the MariaDB database server required for the DVWA application.

Objective:

To confirm that the MariaDB service is running correctly and is ready to accept database connections.

Description:

The MariaDB service was started using `sudo systemctl start mariadb`. Its status was then checked using `sudo systemctl status mariadb`. The terminal shows “**Active: active (running)**” and “**ready for connections**”, confirming that the database server started successfully.

Commands Used:

```
sudo systemctl start mariadb
sudo systemctl status mariadb
```

Result:

MariaDB started successfully and is ready to accept database connections. This completes the database-server startup requirement for the DVWA environment.

➤ MariaDB Database Console Accessed Successfully

Screen shot


```
(jiya@jiya)-[/var/www/html/DVWA]
$ sudo mariadb
Welcome to the MariaDB monitor.  Commands end with ; or \g.
Your MariaDB connection id is 5
Server version: 11.8.8-MariaDB-1 from Debian -- Please help get to 10k stars at https://github.com/MariaDB/Server

Copyright (c) 2000, 2018, Oracle, MariaDB Corporation Ab and others.

Type 'help;' or '\h' for help. Type '\c' to clear the current input statement.

MariaDB [(none)]> █
```

Purpose:

To access the MariaDB database management console for configuring the database required by DVWA.

Objective:

To verify that the MariaDB server accepts administrative connections through the local database console.

Description:

The `sudo mariadb` command was executed in the terminal. The appearance of the **MariaDB [(none)]>** prompt confirms that the MariaDB console was opened successfully and is ready to accept database commands.

Command Used:

```
sudo mariadb
```

Result:

The MariaDB database console was accessed successfully. The system is now ready for the next DVWA database configuration step.

➤ DVWA Database Created Successfully

Screen shot

```
Server version: 11.8.8-MariaDB-1 from Debian -- Please help get to 10k stars a
Copyright (c) 2000, 2018, Oracle, MariaDB Corporation Ab and others.

Type 'help;' or '\h' for help. Type '\c' to clear the current input statement.

MariaDB [(none)]> CREATE DATABASE dvwa;
Query OK, 1 row affected (0.001 sec)

MariaDB [(none)]> █
```

Purpose:

To create the database required for the DVWA application in the MariaDB server.

Objective:

To establish the `dvwa` database that will be used by the DVWA application for storing its application data.

Description:

The `CREATE DATABASE dvwa;` SQL command was executed within the MariaDB console. The successful “Query OK” response confirms that the DVWA database was created successfully.

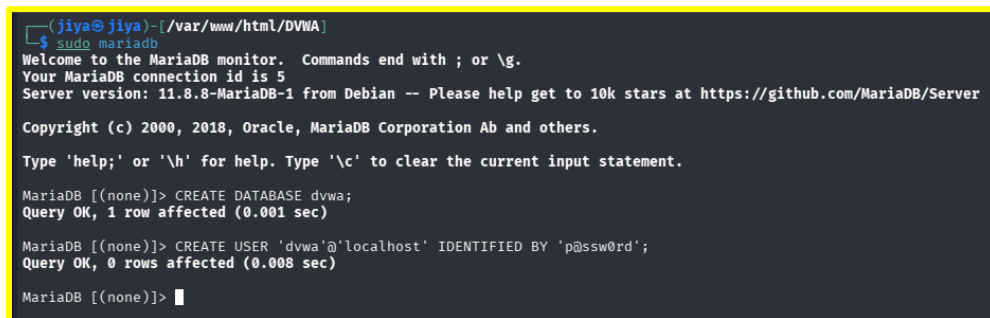
Command Used:

```
CREATE DATABASE dvwa;
```

Result:

The `dvwa` database was successfully created in MariaDB and is ready for the next stage of DVWA database configuration.

➤ **DVWA Database User Created Successfully**

Screen shot

```
(jiya@jiya)-[/var/www/html/DVWA]
$ sudo mariadb
Welcome to the MariaDB monitor.  Commands end with ; or \g.
Your MariaDB connection id is 5
Server version: 11.8.8-MariaDB-1 from Debian -- Please help get to 10k stars at https://github.com/MariaDB/Server

Copyright (c) 2000, 2018, Oracle, MariaDB Corporation Ab and others.

Type 'help;' or '\h' for help. Type '\c' to clear the current input statement.

MariaDB [(none)]> CREATE DATABASE dvwa;
Query OK, 1 row affected (0.001 sec)

MariaDB [(none)]> CREATE USER 'dvwa'@'localhost' IDENTIFIED BY 'p@ssw0rd';
Query OK, 0 rows affected (0.008 sec)

MariaDB [(none)]> █
```

Purpose:

To create a dedicated database user for the DVWA application.

Objective:

To establish the `dvwa` database account that will be used by DVWA to connect to the MariaDB database.

Description:

The `CREATE USER` SQL command was executed in the MariaDB console to create the `dvwa` user for the local host. The successful “Query OK” response confirms that the database user was created successfully.

Command Used:

```
CREATE USER 'dvwa'@'localhost' IDENTIFIED BY 'p@ssw0rd';
```

Result:

The `dvwa` database user was successfully created in MariaDB, preparing the environment for the next database-permission configuration step.

➤ **DVWA Database User Privileges Granted Successfully**

Screen shot

```
MariaDB [(none)]> CREATE USER 'dvwa'@'localhost' IDENTIFIED BY 'p@ssw0rd';  
Query OK, 0 rows affected (0.008 sec)  
  
MariaDB [(none)]> GRANT ALL PRIVILEGES ON dvwa.* TO 'dvwa'@'localhost';  
Query OK, 0 rows affected (0.004 sec)  
  
MariaDB [(none)]> █
```

Purpose:

To provide the DVWA database user with the required privileges to access and work with the `dvwa` database.

Objective:

To grant the `dvwa` user the necessary permissions on the `dvwa` database for proper database connectivity.

Description:

The `GRANT ALL PRIVILEGES` command was executed in the MariaDB console for the `dvwa` database and the local `dvwa` user. The successful “Query OK” response confirms that the required database privileges were granted.

Command Used:

```
GRANT ALL PRIVILEGES ON dvwa.* TO 'dvwa'@'localhost';
```

Result:

The required privileges were successfully granted to the `dvwa` database user, preparing the database environment for the next DVWA configuration step.

➤ MariaDB Privileges Successfully Refreshed

Screen shot

```
Query OK, 0 rows affected (0.004 sec)

MariaDB [(none)]> FLUSH PRIVILEGES;
Query OK, 0 rows affected (0.001 sec)

MariaDB [(none)]> █
```

Purpose:

To refresh the MariaDB privilege tables after configuring the DVWA database user permissions.

Objective:

To ensure that the recently assigned database privileges are reloaded and applied correctly.

Description:

The `FLUSH PRIVILEGES;` command was executed in the MariaDB console. The successful “Query OK” response confirms that the privilege information was refreshed successfully.

Command Used:

```
FLUSH PRIVILEGES;
```

Result:

The MariaDB privileges were successfully refreshed, completing this stage of the DVWA database-user configuration.

➤ DVWA Login Page Successfully Accessed

Screen shot

**Purpose:**

To verify that the DVWA web application is accessible through the local Apache web server and that the login interface is available.

Objective:

To confirm successful access to the DVWA authentication page before proceeding with application configuration and security testing.

Description:

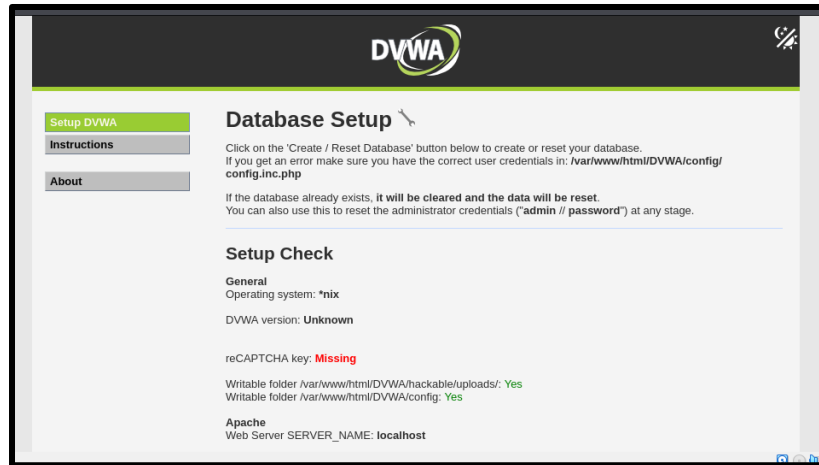
The DVWA application was opened through the local URL `http://localhost/DVWA`. The DVWA login page was displayed successfully, confirming that the application is now accessible through the local web server.

Result:

The DVWA login page was successfully accessed. The application is ready for the next configuration and authentication step.

➤ **DVWA Database Setup and Configuration Check**

Screen shot



Purpose:

To verify the DVWA environment configuration and confirm that the required web server, folders, and permissions are available before creating the DVWA database.

Objective:

To check the DVWA setup requirements and identify any missing configuration items that may affect the application.

Description:

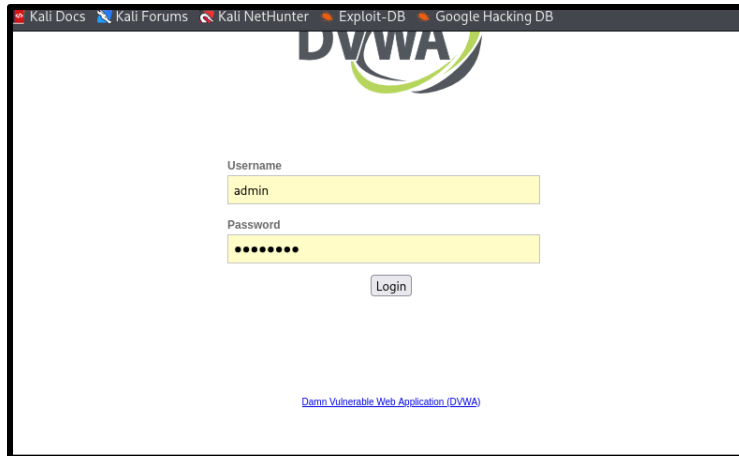
The screenshot shows the DVWA **Database Setup** page. The Setup Check section displays the operating system, DVWA version, reCAPTCHA status, Apache web server information, and writable folder permissions. The screenshot confirms that the required DVWA directories are writable and that Apache is running on the local host. The reCAPTCHA key is shown as missing, which is not required for the basic local DVWA lab setup.

Result:

The DVWA setup environment was successfully accessed and the configuration checks were reviewed. The system is ready to proceed with the **Create / Reset Database** step.

➤ DVWA Login Page After Database Setup

Screen shot

**Purpose:**

To verify that the DVWA application is accessible after completing the database setup.

Objective:

To confirm that the DVWA login interface is available and ready for authentication.

Description:

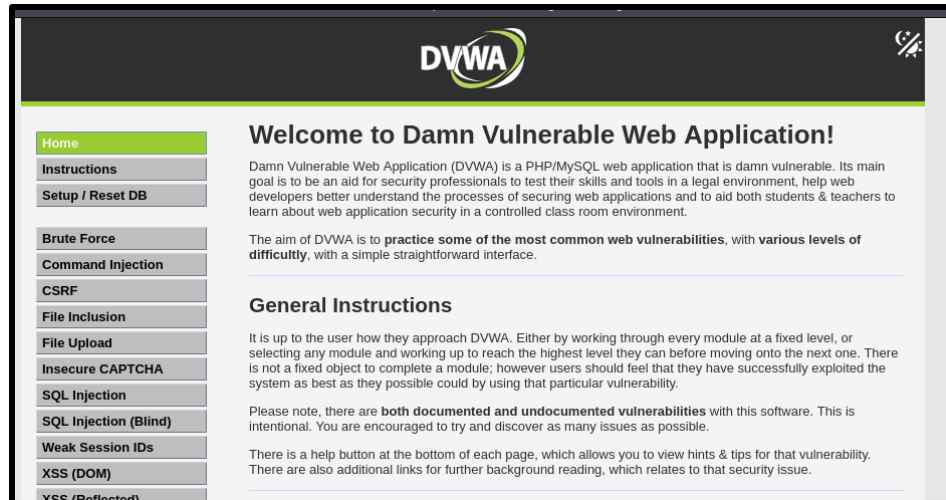
The screenshot shows the DVWA login page at <http://localhost/DVWA/login.php>. The username field contains `admin`, and the password field is prepared for authentication. The Login button is available to proceed into the DVWA application.

Result:

The DVWA login page was successfully accessed, confirming that the application setup was completed sufficiently to proceed with authentication testing.

➤ **Successful Login and DVWA Home Page**

Screen shot



Purpose:

To verify successful authentication and access to the DVWA application.

Objective:

To confirm that the configured DVWA environment is accessible after successful login.

Description:

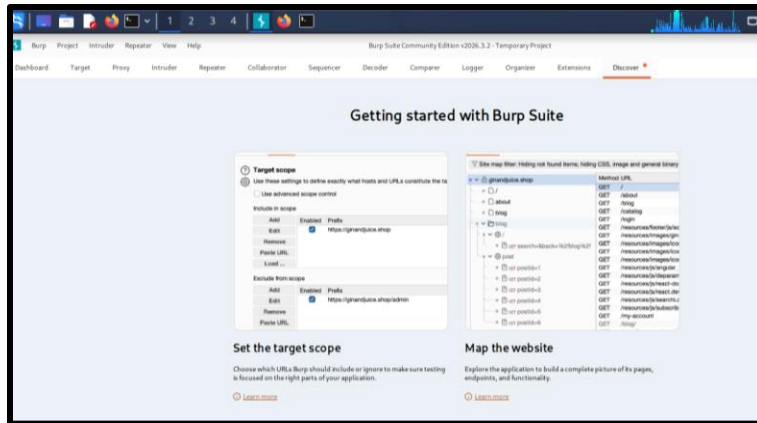
The screenshot shows the DVWA Welcome page after successful authentication. The URL is `http://localhost/DVWA/index.php`. The left-side menu displays different security modules, including Brute Force, Command Injection, CSRF, File Inclusion, SQL Injection, and XSS. This confirms that the DVWA application is functioning correctly and is ready for security testing.

Result:

Successful login was achieved, and the DVWA main dashboard was accessed successfully. The application is ready for the upcoming vulnerability-testing activities.

➤ Burp Suite Community Edition Main Interface

Screen shot



Purpose:

To launch Burp Suite as the web application security testing tool for the authorized DVWA laboratory.

Objective:

To verify that Burp Suite Community Edition is running successfully and ready for configuring the browser proxy and capturing DVWA traffic.

Description:

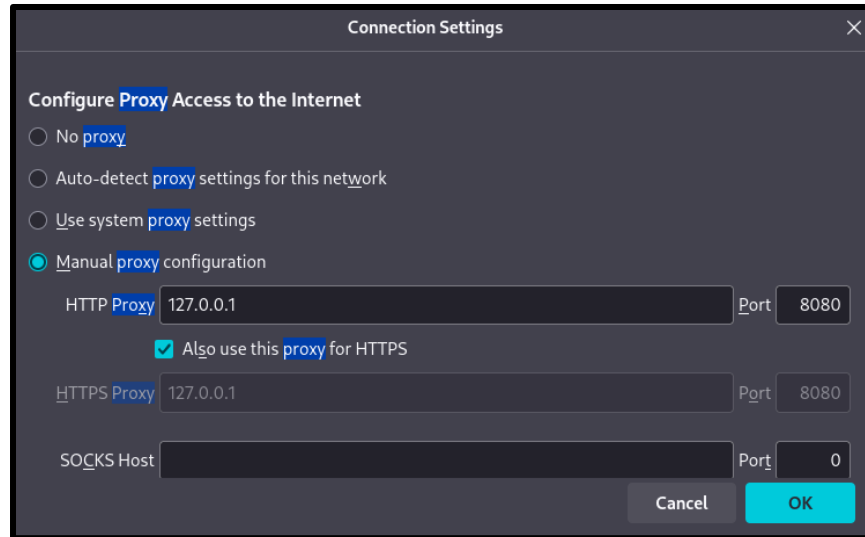
The screenshot shows the Burp Suite Community Edition temporary project interface. The main tabs, including **Dashboard**, **Target**, **Proxy**, **Intruder**, **Repeater**, and **Logger**, are visible. This confirms that Burp Suite has been successfully launched and is ready for the next stage of the controlled DVWA security-testing exercise.

Result:

Burp Suite Community Edition was successfully started and is ready for browser proxy configuration and DVWA request capture. The guide's next steps are browser proxy configuration followed by capturing a DVWA request.

➤ Firefox Manual Proxy Configuration for Burp Suite

Screen shot



Purpose:

To configure Firefox to route web traffic through the Burp Suite proxy in the authorized DVWA testing environment.

Objective:

To establish communication between Firefox and Burp Suite so that DVWA HTTP requests can be captured and analyzed.

Description:

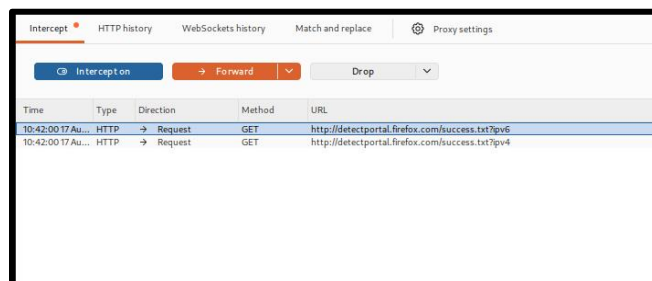
The screenshot shows Firefox **Connection Settings** with the **Manual proxy configuration** option selected. The HTTP proxy is configured as 127.0.0.1 with port 8080, which is the local proxy endpoint used for the Burp Suite testing setup.

Result:

Firefox was configured to use the local Burp Suite proxy, preparing the browser for controlled

➤ Burp Suite HTTP Request Interception Using the Forward Option

Screen shot



Purpose:

To demonstrate that Burp Suite is successfully intercepting HTTP requests sent by the Firefox browser.

Objective:

To verify the communication between the browser and DVWA through the Burp Suite proxy.

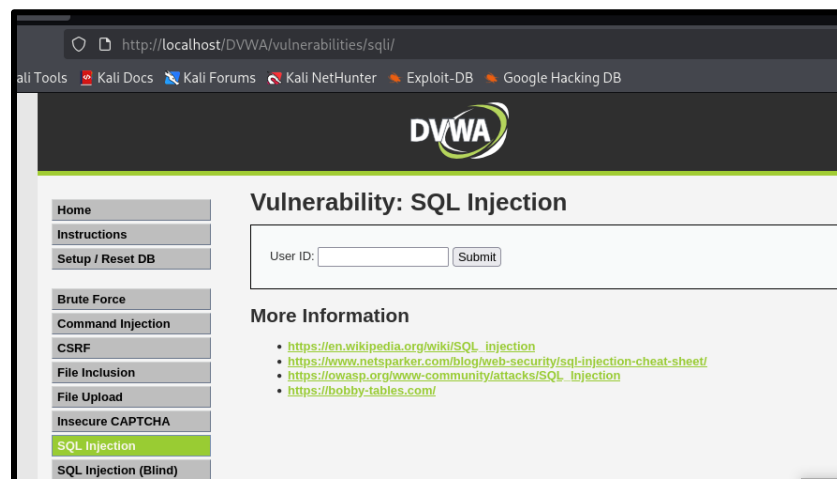
Description:

This screenshot shows Burp Suite's Proxy Intercept feature with an intercepted HTTP request. The request is temporarily paused by Burp Suite, allowing the tester to inspect it before forwarding it to the destination server. The **Forward** button is used to send the intercepted request onward.

Result:

The HTTP request was successfully intercepted by Burp Suite, confirming that the browser traffic is passing through the configured proxy.

➤ **DVWA SQL Injection Testing Interface**

Screen shot**Purpose:**

To access the DVWA SQL Injection module and prepare the authorized lab environment for SQL Injection security testing.

Objective:

To verify that the SQL Injection testing interface is available and ready for controlled vulnerability assessment within the DVWA environment.

Description:

The DVWA SQL Injection module was opened from the application menu after successful authentication. The page provides the input interface required for testing how user-supplied data is processed by the application. This module is intentionally vulnerable and is used within the controlled DVWA laboratory to study SQL Injection behavior.

Observation:

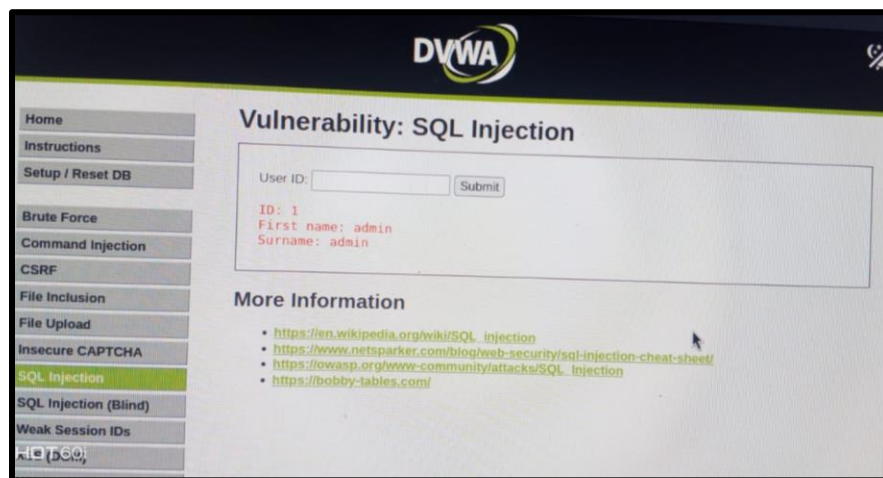
The SQL Injection testing interface was successfully loaded and is ready for the next testing stage.

Result:

The DVWA SQL Injection module was successfully accessed, confirming that the application is ready for the authorized SQL Injection assessment.

➤ SQL Injection Baseline Test Result

Screen shot



Observation:

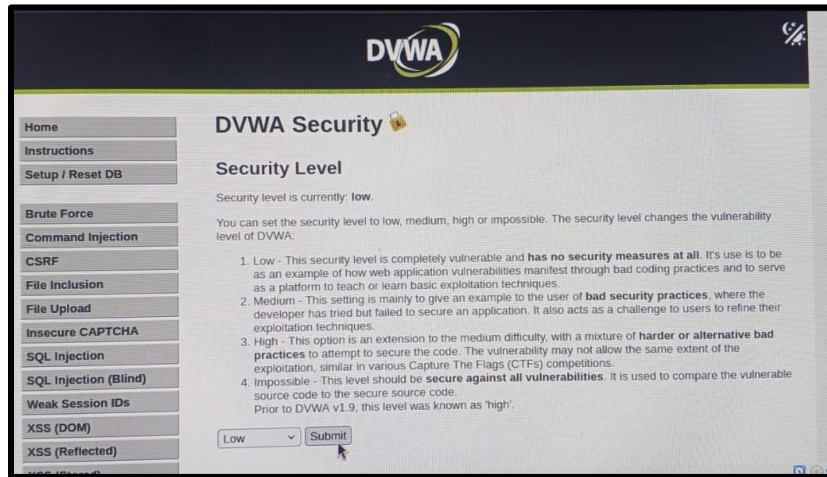
A normal User ID value of 1 was submitted through the DVWA SQL Injection interface. The application returned the corresponding record, showing **admin** as both the first name and surname.

Result:

The normal application response was successfully recorded as a baseline for the subsequent controlled SQL Injection assessment.

➤ DVWA Security Level Configuration

Screen shot



Purpose:

To configure the DVWA application to the Low security level for the controlled SQL Injection laboratory exercise.

Objective:

To establish the intended vulnerable testing configuration required to demonstrate SQL Injection behavior in the DVWA environment.

Description:

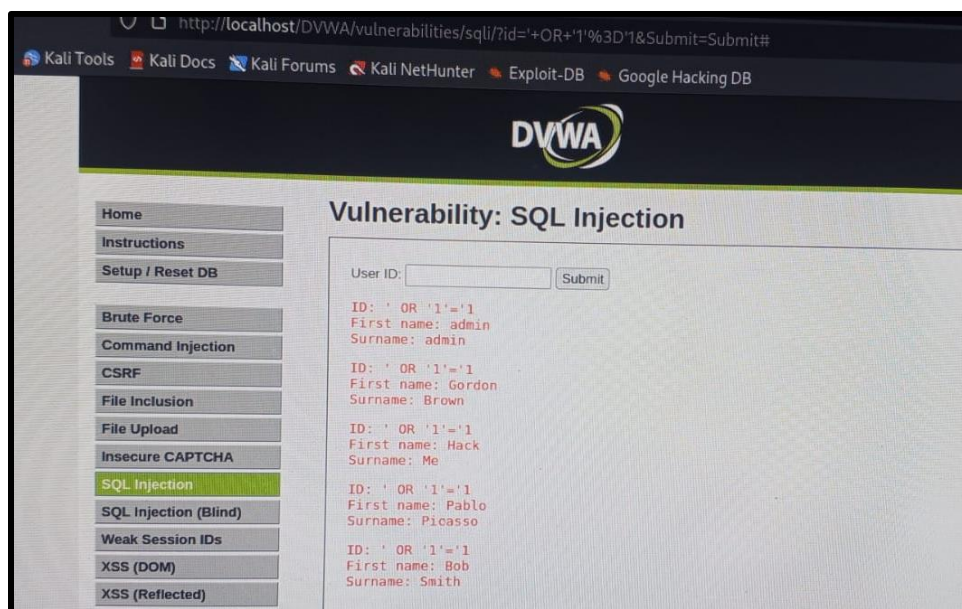
The DVWA Security page was opened and the security level was configured to **Low**. This setting provides the intentionally vulnerable configuration required for the authorized security-testing exercise.

Result:

The DVWA security level was successfully configured to Low, preparing the application for the next SQL Injection test.

➤ Successful SQL Injection Test Result

Screen shot

**Purpose:**

To demonstrate the SQL Injection vulnerability in the authorized DVWA laboratory environment.

Objective:

To determine whether manipulated user input can alter the application's database query and return records beyond the intended User ID.

Description:

The SQL Injection test was performed against the DVWA SQL Injection module using the controlled test input. The application returned multiple database records instead of a single record associated with a normal User ID. The result includes several user entries, demonstrating that the input affected the application's database query.

Observation:

The application returned multiple user records, including different first names and surnames. This behavior indicates that the User ID input is being processed unsafely and that the SQL query can be manipulated.

Impact:

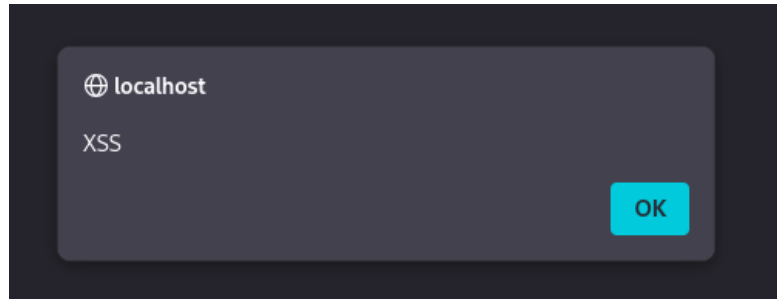
An attacker could potentially bypass intended input restrictions and retrieve unauthorized database records if the same weakness existed in a real application.

Severity: High**Result:**

The SQL Injection vulnerability was successfully demonstrated in the controlled DVWA environment.

➤ Successful XSS Alert Popup

Screen shot



Purpose:

To verify whether the DVWA application executes the submitted XSS test input.

Objective:

To demonstrate the execution of a Reflected XSS test through a controlled input.

Description:

The Reflected XSS page was tested by submitting the test input in the input field. After submission, the browser displayed an “XSS” **alert popup**, indicating that the input was interpreted and executed by the application.

Result:

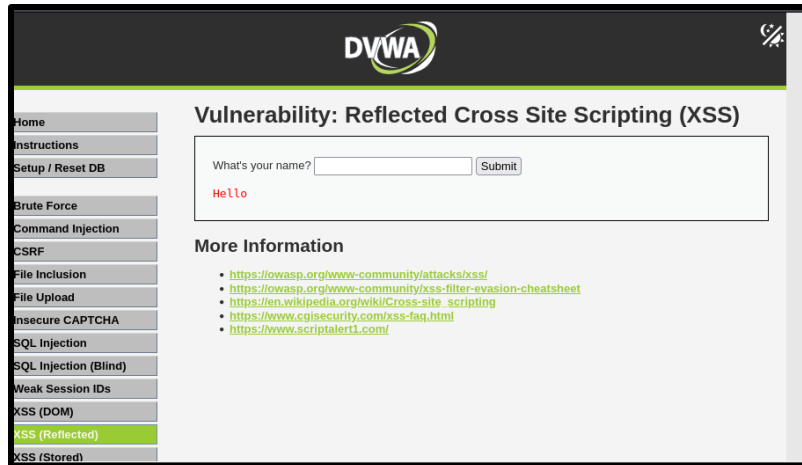
The XSS alert was successfully displayed, confirming that the Reflected XSS test was successful at the selected DVWA security level.

Conclusion:

The result demonstrates the presence of a Reflected XSS vulnerability in the DVWA test environment.

➤ Reflected Output After XSS Test

Screen shot

**Purpose:**

To observe the application response after closing the XSS alert.

Objective:

To verify the output generated by the application after successful execution of the XSS test.

Description:

After the “XSS” alert popup was displayed, the **OK** button was selected. The DVWA page then displayed “Hello” in the output section, showing the response generated after processing the submitted input.

Result:

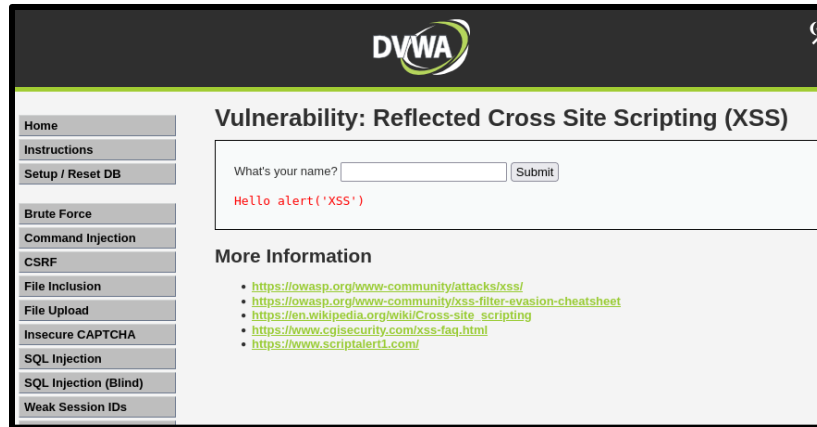
The expected output “Hello” was displayed successfully after the alert was closed.

Conclusion:

The result confirms the complete flow of the Reflected XSS test, from input submission and alert execution to the resulting page output.

➤ **XSS Test Result at Medium Security Level**

Screen shot

**Purpose:**

To test the Reflected XSS functionality in DVWA at the **Medium** security level.

Objective:

To observe how the application handles the submitted XSS test input when Medium security controls are enabled.

Description:

The DVWA security level was changed to **Medium**. The **XSS (Reflected)** module was opened, and the test input was entered in the “**What is your name?**” field. After clicking **Submit**, the application processed the input and displayed the resulting output directly on the page.

Result:

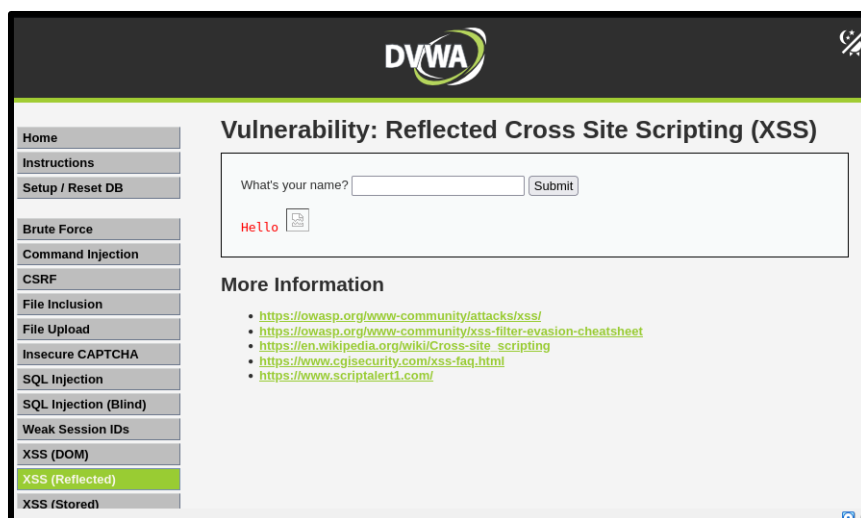
The input was processed and the result appeared below the input field. **No XSS alert popup was displayed** at this stage.

Conclusion:

The result shows that the application handled the submitted input differently at the Medium security level compared with the previous Low-level test.

➤ **XSS Test Using Image Event Handler**

Screen shot

**Purpose:**

To test another Reflected XSS payload at the Medium security level.

Objective:

To determine whether an alternative XSS payload using an image event handler can be executed by the DVWA application.

Description:

The second XSS test payload was entered into the “**What is your name?**” field and submitted. The application processed the input and displayed “**Hello**” along with a small image/box icon. An XSS alert was also triggered during the test.

Result:

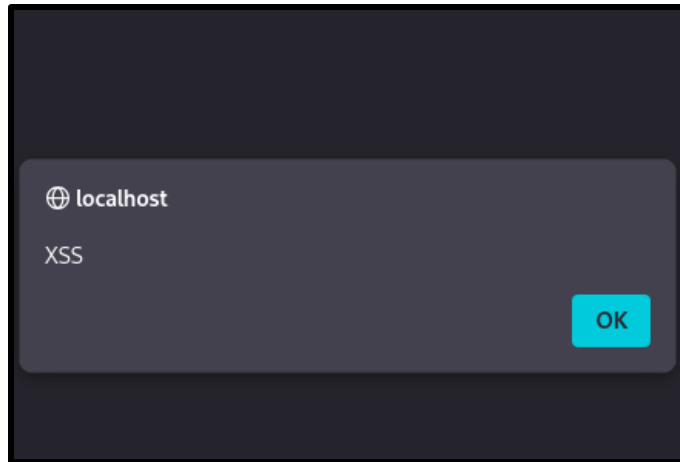
The alternative XSS payload was successfully processed, and the **XSS alert** appeared. The page then displayed the **Hello** output with the image/box indicator.

Conclusion:

The test demonstrates that alternative client-side event-handler based XSS input can be executed in the controlled DVWA environment. The payload used is one of the XSS testing examples provided in the project guide.

➤ **Successful XSS Execution Using SVG Event Handler**

Screen shot

**Purpose:**

To test Reflected XSS using an alternative SVG-based event-handler payload.

Objective:

To determine whether the DVWA application executes the SVG-based XSS test input.

Description:

The third XSS test payload was entered into the “**What is your name?**” field at the Medium security level. After submitting the input, the browser displayed an “**XSS**” **alert popup** from the local DVWA application.

Result:

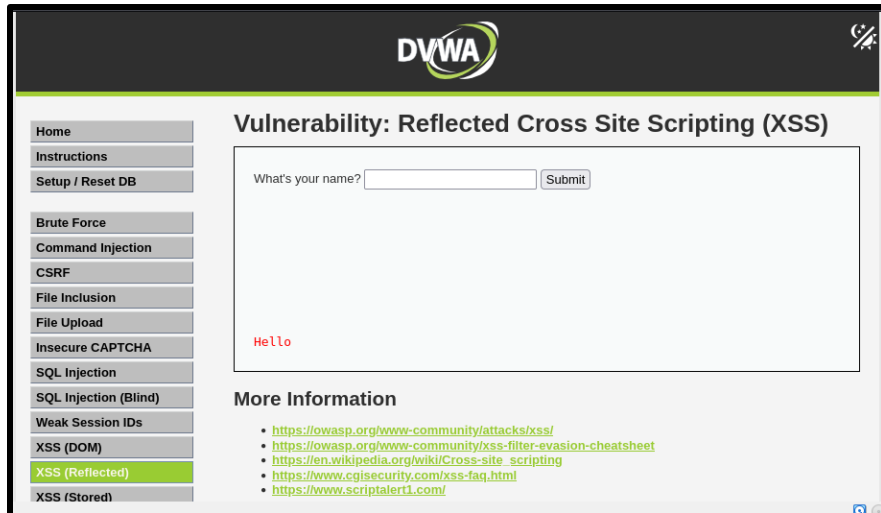
The **XSS alert** was **successfully triggered**, confirming that the tested SVG event-handler input was executed.

Conclusion:

The test demonstrates that alternative XSS input patterns can execute in the controlled DVWA environment.

➤ **XSS Test Page Output After Alert**

Screen shot

**Purpose:**

To observe the application response after the XSS alert was dismissed.

Objective:

To verify the resulting page output following successful execution of the SVG-based XSS test.

Description:

After the “XSS” alert popup was displayed, the **OK** button was selected. The DVWA page then returned to the normal output view and displayed “Hello” along with the observed spacing/output.

Result:

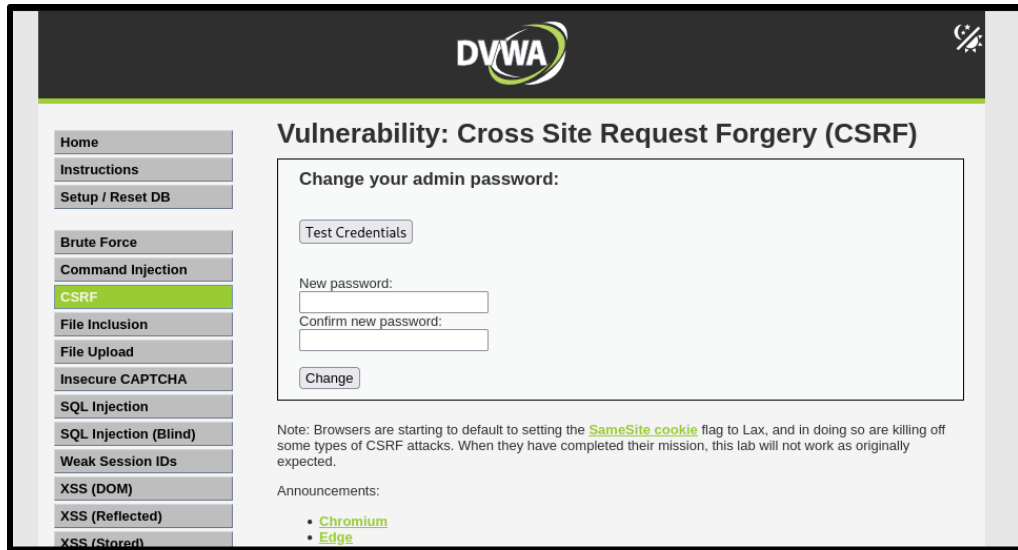
The **Hello** output was successfully displayed after closing the XSS alert.

Conclusion:

The result completes the XSS testing sequence by showing the application's response after the alert execution.

➤ **CSRF Vulnerability Testing Interface**

Screen shot



Purpose:

To access the DVWA CSRF module and establish a baseline for CSRF vulnerability testing at the Medium security level.

Objective:

To examine the password-change functionality and determine whether the application provides protection against unauthorized cross-site requests.

Description:

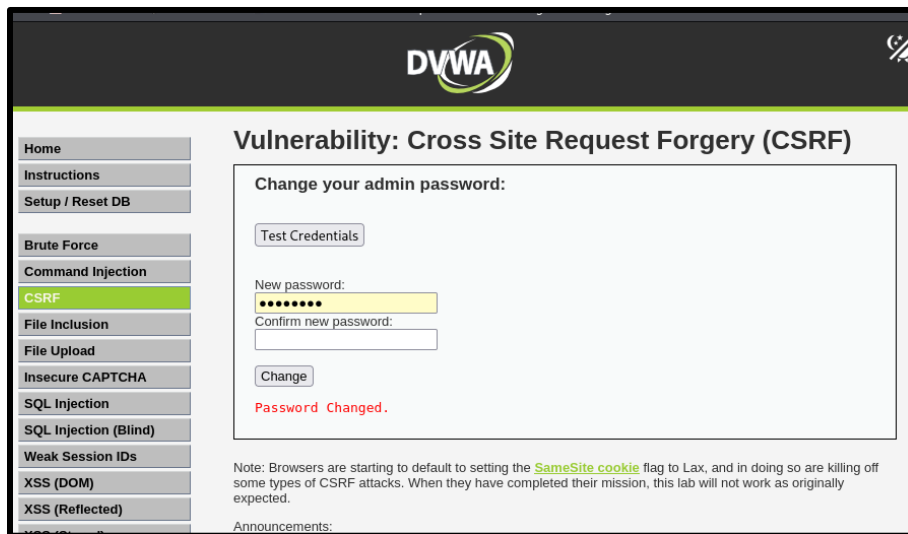
The screenshot shows the DVWA **Cross-Site Request Forgery (CSRF)** module at **Medium security level**. The page contains fields for entering and confirming a new administrator password, along with a **Change** button. This interface is used as the starting point for the CSRF security test.

Result:

The CSRF module was successfully accessed at Medium security level, and the password-change interface was available for further testing.

➤ Successful CSRF Password Change

Screen shot

**Purpose:**

To demonstrate a successful Cross-Site Request Forgery (CSRF) attack scenario in DVWA by submitting a password-change request.

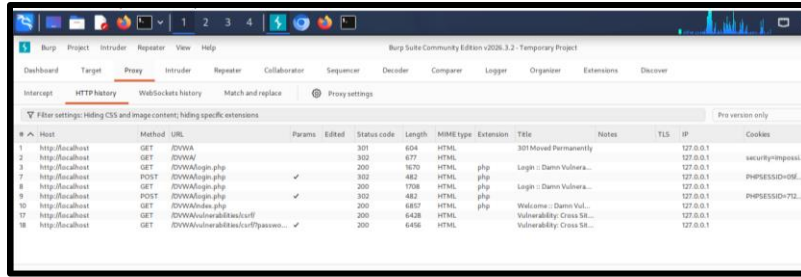
Observation: The password-change request was successfully processed in the DVWA environment. However, the captured request also contained a `user_token` parameter, so the available evidence does not conclusively demonstrate a complete unauthorized cross-site request attack.

Result:

The CSRF password-change functionality was successfully tested, but a complete unauthorized CSRF exploit could not be conclusively demonstrated.

➤ **CSRF Request Captured in Burp Suite HTTP History**

Screen shot



Purpose:

To verify that the DVWA CSRF page request is successfully intercepted and recorded by Burp Suite.

Objective:

To confirm that Burp Suite is properly configured to monitor HTTP traffic generated by the DVWA application.

Description:

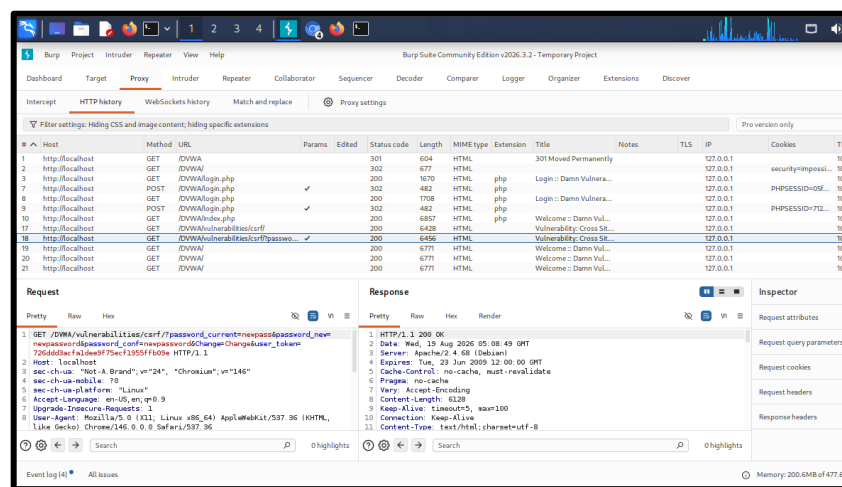
The screenshot shows Burp Suite **Proxy** → **HTTP history**, where requests to the DVWA application are visible. The `/DVWA/vulnerabilities/csrf/password...` request confirms that the CSRF page traffic is being captured successfully.

Result:

The HTTP request was successfully captured in Burp Suite, confirming that the proxy is working correctly and the DVWA CSRF traffic can be analyzed.

➤ CSRF Password Change Request Captured in Burp Suite

Screen shot



Purpose:

To capture and analyze the HTTP request generated when changing the DVWA password.

Evidence:

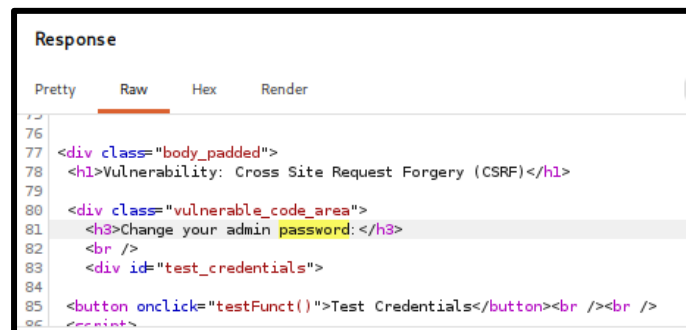
Burp Suite HTTP History shows a GET request to the DVWA CSRF password-change endpoint. The request contains the parameters `password_new`, `password_conf`, `Change`, and `user_token`.

Observation:

The password-change operation is performed through a GET request with password-related parameters visible in the request URL. A `user_token` parameter is also present, indicating that the application uses a token as part of its CSRF protection mechanism.

Result: CSRF password-change functionality was tested and the request was captured in Burp Suite. A `user_token` was present in the request, therefore the available evidence is insufficient to conclusively confirm an exploitable CSRF vulnerability.

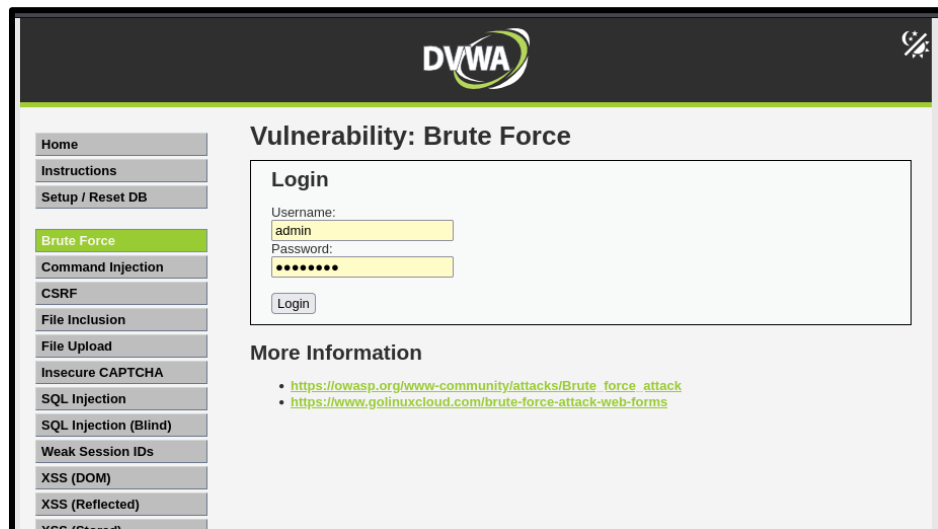
➤ **DVWA Password Change Page Displayed in Burp Suite Response**

Screen shot

- **Purpose:** To verify the response returned after accessing the password-change functionality.
- **Evidence:** The rendered response displays the “Change your admin password” page.
- **Observation:** The response confirms that the password-change functionality is accessible through the captured request.
- **Result:** The DVWA password-change page was successfully returned by the server.

➤ DVWA Authentication / Brute Force Testing Interface

Screen shot



Purpose:

To access the DVWA authentication-testing module and observe the interface used for evaluating authentication security.

Objective:

To examine the available username and password authentication fields within the controlled DVWA environment.

Description:

The DVWA Brute Force module was opened after successful authentication. The page provides username and password input fields along with a login option, allowing authentication behavior to be examined within the intentionally vulnerable testing application.

Observation:

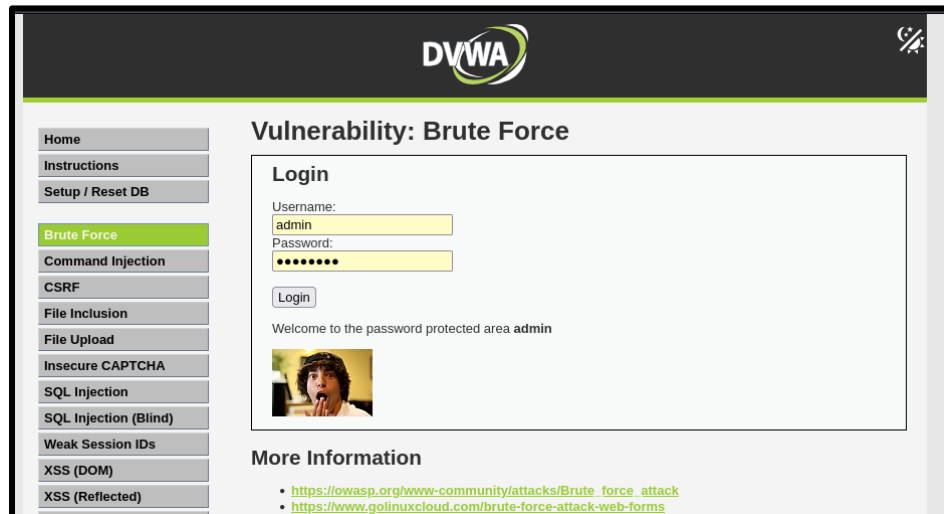
The authentication interface accepts user-supplied username and password values through the web form. This provides a basis for observing how the application handles authentication requests.

Result:

The DVWA authentication-testing interface was successfully accessed and documented, completing the initial authentication-security observation stage.

➤ Successful Brute Force Login Result

Screen shot



Purpose

The purpose of this test was to test login authentication in the DVWA Brute Force module and verify that the password-protected area could be accessed using valid credentials.

Objective

- To test username- and password-based authentication.
- To observe successful authentication using valid credentials.
- To analyze the login response in the context of the Brute Force vulnerability.

Description

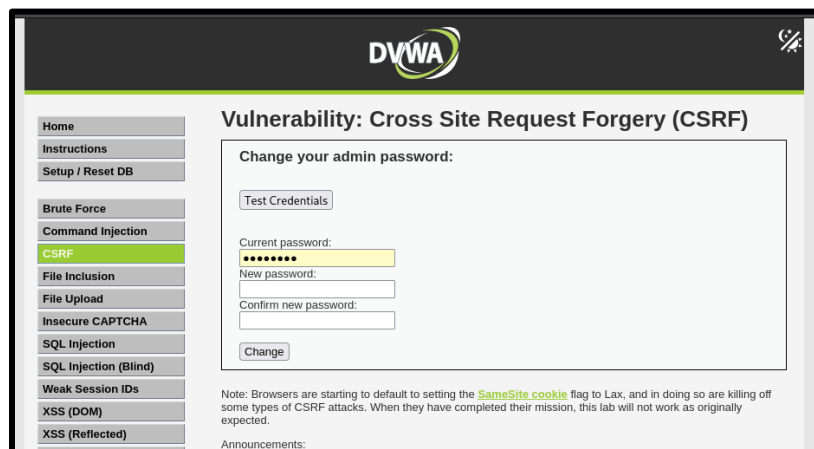
On the **Vulnerability: Brute Force** page in DVWA, the username `admin` was entered in the username field, and the password was provided. The **Login** button was then selected. After successful authentication, the application displayed the message **“Welcome to the password protected area admin.”** This confirmed that the provided credentials were accepted and that the protected area was accessible.

Result

The login was successfully authenticated. DVWA displayed the password-protected area for the **admin** user. This screenshot provides evidence of successful authentication.

➤ CSRF Page with Pre-filled Current Password

Screen shot



Purpose:

To observe the CSRF password-change form and verify how the **Current Password** field is handled when the page is loaded.

Objective:

- CSRF password-change functionality ko observe karna.
- Check karna ke Current Password field automatically populated hai ya nahi.
- Password-change form ke input fields aur security behavior ko document karna.

Description:

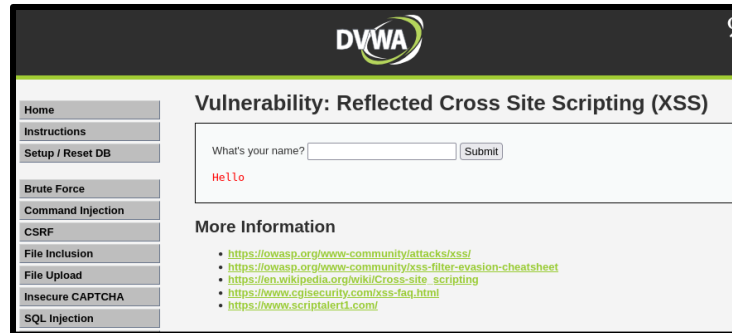
The screenshot shows the DVWA **CSRF vulnerability page** where the **Current Password** field is already populated automatically. The page also contains the fields required for changing the password. This screenshot was captured as evidence of the initial state of the CSRF password-change form before performing further testing.

Result:

The Current Password field was found **pre-filled automatically** when the CSRF page was opened. This behavior was documented as part of the CSRF testing process.

➤ Reflected Cross-Site Scripting (XSS) Testing

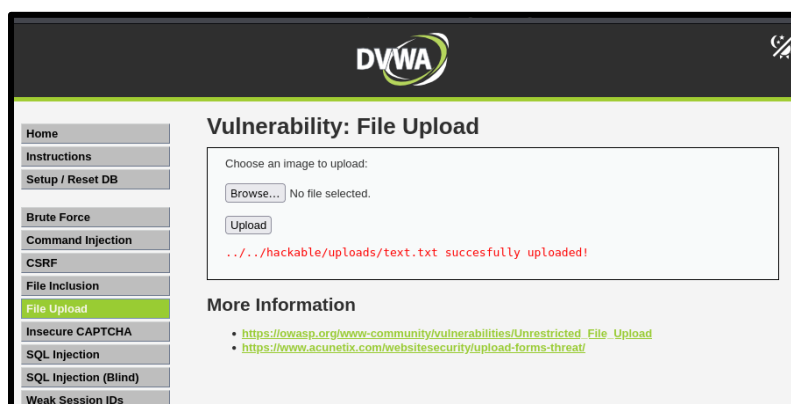
Screen shot



- **Vulnerability:** Reflected Cross-Site Scripting (Reflected XSS)
- **Test Payload:** `<script>alert('XSS')</script>`
- **Evidence:** Application ne submitted input ko response mein reflect kiya aur JavaScript alert trigger hua.
- **Observed Output:** Hello
- **Observation:** User-supplied input was reflected in the server response without adequate output encoding, allowing the XSS payload to execute.
- **Result:** **Reflected XSS confirmed.**
- **Risk:** Malicious script content could execute in a victim's browser when they interact with a crafted request.
- **Recommendation:** Apply proper input validation and context-aware output encoding before reflecting user-controlled data.

➤ File Upload Vulnerability – Successful File Upload

Screen shot



Purpose:

To test whether the DVWA application accepts and uploads a user-supplied file without sufficient file-type validation.

Testing:

A normal text file named `text.txt` containing `File upload test` was selected through the file upload functionality and submitted.

Evidence:

The application displayed the message:

```
../../hackable/uploads/text.txt successfully uploaded!
```

Observation:

The application successfully accepted the `.txt` file and stored it in the `hackable/uploads/` directory. This demonstrates that the file upload functionality permits this file type.

Finding:

The file upload mechanism does not appear to restrict the tested `.txt` file type at the current DVWA security level.

Risk:

Medium — unrestricted or insufficiently validated file uploads can become a security risk if executable or malicious file types are also accepted.

Impact:

If dangerous file types can be uploaded and executed by the server, an attacker could potentially compromise application data or server functionality.

Recommendation:

Implement strict server-side file validation, allow only required file extensions and MIME types, rename uploaded files, store uploads outside the executable web directory, and apply appropriate access controls.

Result:

The file upload test was **successful**, confirming that the application accepted and stored the submitted `text.txt` file.

➤ **Security Findings and Recommendations**

Finding: Brute Force Authentication**Evidence:**

DVWA Brute Force module mein valid username and password credentials ke through successful login observe kiya gaya. Burp Suite ke HTTP requests/responses ko bhi review kiya gaya.

Observation:

The authentication mechanism allowed repeated login attempts without sufficient protection against automated or repeated password guessing.

Risk:

Medium to High — An attacker may repeatedly attempt different passwords against an account.

Impact:

Successful brute-force attacks could result in unauthorized access to user accounts and exposure of protected information.

Recommendation:

Implement account lockout or temporary delays after multiple failed attempts, use rate limiting, enforce strong passwords, and consider multi-factor authentication.

➤ **Finding: SQL Injection**

Evidence:

DVWA SQL Injection module mein specially crafted input ke through database query behavior observe kiya gaya.

Observation:

User-supplied input was not adequately protected from being interpreted as part of a database query.

Risk:

High — SQL Injection can allow unauthorized interaction with database queries.

Impact:

An attacker may potentially access, modify, or disclose database information.

Recommendation:

Use prepared statements/parameterized queries, validate and sanitize input, apply least-privilege database accounts, and avoid directly concatenating user input into SQL queries.

➤ **Finding: Cross-Site Scripting (XSS)**

Evidence:

DVWA ke XSS modules mein user-controlled input ko browser ke response mein process hote observe kiya gaya.

Observation:

The application did not sufficiently validate or encode user-supplied input before displaying it.

Risk:

Medium — Malicious scripts may execute in the context of the vulnerable application.

Impact:

XSS may affect users who interact with malicious input and can potentially expose session-related information or alter page content.

Recommendation:

Apply proper output encoding, validate input, use context-appropriate sanitization, and implement a strong Content Security Policy (CSP).

➤ **Finding: Cross-Site Request Forgery (CSRF)**

Evidence:

The DVWA password-change functionality was examined using Burp Suite to observe the HTTP request and security controls.

Observation:

The application demonstrated weaknesses in protecting sensitive state-changing requests against unauthorized cross-site requests.

Risk:

Medium — A vulnerable request may be triggered without sufficient verification of the user's intent.

Impact:

An attacker could potentially cause an authenticated user to perform an unwanted action.

Recommendation:

Use unique and unpredictable anti-CSRF tokens for state-changing requests, validate the Origin/Referer headers where appropriate, and use suitable SameSite cookie settings.

➤ **Finding: Session and Authentication Security**

Evidence:

Authentication-related requests and responses were reviewed during testing through DVWA and Burp Suite.

Observation:

The lab demonstrated common weaknesses related to authentication and session security controls.

Risk:

Medium

Impact:

Weak authentication or session controls can increase the possibility of unauthorized access.

Recommendation:

Use secure session management, HTTPS, appropriate cookie flags such as Secure and HttpOnly, session expiration, strong authentication controls, and rate limiting.

➤ **Overall Recommendation**

The tested vulnerabilities demonstrate the importance of secure input validation, parameterized database queries, strong authentication controls, CSRF protection, secure session management, and proper output encoding. These controls should be implemented together as part of a secure software development lifecycle.

➤ **Final Security Summary and Conclusion**

Security Summary

The DVWA security assessment was conducted in a controlled Kali Linux laboratory environment to identify and understand common web application security vulnerabilities. During the assessment, different vulnerability categories were examined, including Brute Force, SQL Injection, Cross-Site Scripting (XSS), and Cross-Site Request Forgery (CSRF). Burp Suite was also used to observe and analyze HTTP requests and responses during security testing.

The assessment demonstrated that insecure input handling, weak authentication controls, insufficient request validation, and inadequate security mechanisms can create significant risks for web applications. The vulnerabilities observed could potentially affect the confidentiality, integrity, and availability of application data if similar weaknesses existed in a real-world application.

➤ **Conclusion**

This project provided practical experience in identifying and analyzing common web application vulnerabilities. The testing process showed how attackers may take advantage of weaknesses in authentication, input validation, database queries, browser-side input processing, and state-changing requests.

The assessment also highlighted the importance of implementing security controls such as prepared statements, input validation, output encoding, CSRF tokens, rate limiting, secure session management, strong authentication, and least-privilege access.

Overall, the DVWA project improved practical understanding of web application security and demonstrated the importance of identifying vulnerabilities early and applying appropriate remediation techniques. The findings and recommendations documented in this report can help developers build more secure and resilient web applications.

